BTS SIO · OPTION SISR
Cybersécurité · Travaux pratiques guidés



SÉANCE 3

Atelier Kali 1

Découverte de données exposées

Cours détaillé et progressif — tutoriel pas à pas

Concepts · lecture de trames Wireshark · manipulations commentées
Définitions · règles de cybersécurité · analyse · exercices corrigés

Nmap **Nikto** **Gobuster** **Wireshark** **ExifTool**

Laboratoire isolé : VirtualBox · Kali Linux · Ubuntu Server · OWASP Juice Shop

Document pédagogique — toute manipulation est limitée aux machines virtuelles du laboratoire (192.168.56.0/24).

Sommaire

Comment utiliser ce support . . . 4

Partie 1 — Comprendre . . . 6

1.1 Un réseau, des hôtes, des adresses IP . . . 6
1.2 L'adresse MAC . . . 6
1.3 Le modèle en couches et l'encapsulation . . . 7
1.4 Port, service, application . . . 7
1.5 TCP, UDP et ICMP . . . 8
1.6 Bannière, version, HTTP en clair . . . 8
1.7 Reconnaissance, recherche de vulnérabilités, exploitation . . . 9
1.8 Le vocabulaire de l'analyse . . . 9
1.9 Architecture du laboratoire . . . 10

Partie 2 — Mode d'emploi : lire une trame dans Wireshark . . . 11

2.1 Le principe : lire de l'extérieur vers l'intérieur . . . 11
2.2 Anatomie d'une trame Ethernet II . . . 11
2.3 Anatomie d'un paquet IPv4 . . . 12
2.4 Les zones de l'interface Wireshark . . . 13
2.5 La méthode en 10 étapes . . . 14
2.6 Cas n°1 — Lire un ping (ICMP), octet par octet . . . 14
2.7 Cas n°2 — Diffusions et trames inconnues . . . 17
2.8 Cas n°3 — Lire une requête DNS (UDP) . . . 19
2.9 Cas n°4 — Des encapsulations plus profondes . . . 19
2.10 Cas n°5 — La diffusion expliquée par un schéma . . . 20
2.11 Convertir : hexadécimal, décimal, binaire . . . 21
2.12 Les filtres Wireshark . . . 22
2.13 Les erreurs d'interprétation les plus fréquentes . . . 22
2.14 Fiche de lecture d'une trame (à recopier) . . . 23
2.15 Exercices de lecture de trames . . . 23

Partie 3 — Installer le laboratoire . . . 25

3.1 Prérequis matériels et logiciels . . . 25
3.2 Choisir le mode réseau . . . 25
3.3 Installer la machine Kali Linux . . . 26
3.4 Préparer la cible Ubuntu Server . . . 27
3.5 Configurer l'adresse de Kali . . . 29
3.6 Vérifier la communication, couche par couche . . . 29
3.7 Isoler le laboratoire et prendre un instantané . . . 30

3.8 Diagnostic des problèmes courants . . . 30

Partie 4 — Pratiquer : Nmap, Nikto, Gobuster . . . 31

4.1 Nmap — identifier les ports et les services . . . 31
4.2 Nikto — vérifier le serveur web . . . 34
4.3 Gobuster — découvrir des chemins web . . . 35

Partie 5 — Pratiquer : capture et métadonnées . . . 38

5.1 Wireshark en situation : capturer un échange HTTP . . . 38
5.2 ExifTool : les métadonnées . . . 40

Partie 6 — Analyser et rédiger . . . 42

6.1 De l'observation à la recommandation . . . 42
6.2 Évaluer un risque . . . 42
6.3 Distinguer un faux positif : la démarche . . . 42
6.4 Rédiger le rapport . . . 43
6.5 Grille d'évaluation . . . 43

Partie 7 — Appliquer : exercices . . . 44

7.1 Exercices guidés . . . 44
7.2 Exercices autonomes . . . 45

Partie 8 — Réviser . . . 46

8.1 Récapitulatif des règles de cybersécurité . . . 46
8.2 Quiz corrigé . . . 46
8.3 Lexique . . . 48
8.4 Fiche mémo des commandes . . . 49
8.5 FAQ . . . 49
8.6 Documentation . . . 49

Comment utiliser ce support

Ce document remplace la fiche de travaux pratiques initiale par un véritable **cours-tutoriel**. Il conserve les mêmes outils (Nmap, Nikto, Gobuster, Wireshark, ExifTool), le même laboratoire isolé (VirtualBox, Kali, Ubuntu Server, OWASP Juice Shop) et le même objectif : **comprendre ce que l'on fait, pourquoi on le fait, et comment interpréter le résultat**.

La séance en présentiel dure 2 heures : elle sert à **pratiquer**. Les explications de ce support sont à lire avant la séance (parties 1 et 2) et après (parties 6 à 8).

Partie	Contenu	Durée indicative
1. Comprendre	Réseau, IP, MAC, ports, services, vocabulaire de l'analyse	30 min de lecture
2. Lire une trame	Mode d'emploi complet de Wireshark, cas commentés, exercices	60 min
3. Installer	Laboratoire VirtualBox, Kali, Ubuntu, Juice Shop	45 min
4. Pratiquer (réseau et web)	Nmap, Nikto, Gobuster, curl	90 min
5. Pratiquer (capture et métadonnées)	Wireshark en situation, ExifTool	60 min
6. Analyser et rédiger	Observation, preuve, faux positif, risque, rapport type, grille d'évaluation	45 min
7. Appliquer	Exercices guidés, exercices autonomes, corrigés	60 min
8. Réviser	Quiz corrigé, lexique, FAQ, documentation	30 min

Logique de chaque chapitre : notion théorique → schéma → démonstration → explication des résultats → exercice → corrigé.

Les encadrés utilisés

À RETENIR
L'idée essentielle du chapitre.

DÉFINITION Terme
Une définition à connaître par cœur.

RÈGLE DE CYBERSÉCURITÉ Titre de la règle
Une règle de cybersécurité (numérotée R1, R2…) : elle est récapitulée en partie 8.

PIÈGE
Une erreur fréquente à éviter.

ATTENDU
Ce que vous devez observer à l'écran si tout fonctionne.

NE PAS CONCLURE
Ce que l'observation ne démontre pas.

Lire les consoles simulées

Toutes les commandes de ce document sont présentées dans une **console simulée** (fond noir). Le code couleur est toujours le même :

légende des couleurs

```
kali@kali:~$ sudo nmap -sV -T3 -p 22,80,3000 192.168.56.20 > ~/atelier-kali-1/nmap/scan.txt
# ceci est un commentaire explicatif (gris italique) : il n'est pas à taper
22/tcp   open   ssh        OpenSSH 9.x
80/tcp   closed http
Note : texte jaune = avertissement ou état intermédiaire
```

Couleur	Signification
vert gras (`kali@kali`)	Invite : utilisateur@machine. Le chemin est en bleu, puis le `$`
blanc gras	Nom du programme lancé (`nmap`, `curl`…) ; `sudo` en rouge
cyan	Options (`-sV`, `-p`…)
violet / vert clair / ambre	Adresses IP et URL / chemins de fichiers / chaînes entre guillemets
gris clair	Résultat affiché par la machine
vert / rouge / jaune	Réussite, port ouvert / échec, port fermé / avertissement, port filtré

CADRE LÉGAL ET ÉTHIQUE **Scanner sans autorisation est une infraction**

Scanner, sonder ou capturer le trafic d'un système sans autorisation est une infraction pénale en France (articles 323-1 et suivants du Code pénal : accès ou maintien frauduleux dans un système de traitement automatisé de données, entrave, etc.). Dans cet atelier :

- vous ne ciblez que les machines virtuelles du laboratoire (plage privée `192.168.56.0/24`) ;
- vous n'envoyez aucun vrai mot de passe ni aucune donnée personnelle réelle ;
- vous ne capturez jamais le trafic d'un tiers (réseau du lycée, Wi-Fi public, famille) ;
- les captures et résultats restent dans le laboratoire.

RÈGLE R1 **Autorisation préalable et périmètre**

On ne teste **que** ce que l'on est explicitement autorisé à tester, dans un périmètre défini (adresses, dates, méthodes). En situation professionnelle, cette autorisation est écrite et signée par le responsable du système.

RÈGLE R2 **Données fictives et confidentialité**

On n'utilise que des données fictives en laboratoire. Une capture réseau peut contenir des identifiants ou des données personnelles : elle est traitée comme une donnée sensible (RGPD) — jamais publiée, jamais partagée hors du cadre pédagogique.

PARTIE 1

Comprendre

30 min de lecture

1

OBJECTIF
Acquérir le vocabulaire et les mécanismes réseau indispensables pour comprendre ce que font les outils et ce qu'affichent leurs résultats.

1.1 Un réseau, des hôtes, des adresses IP

DÉFINITION Hôte
Tout équipement connecté à un réseau : ordinateur, serveur, imprimante, téléphone, machine virtuelle.

Pour que deux hôtes se parlent, chacun possède une **adresse IP**. Une adresse IPv4 est un nombre de **32 bits**, écrit en quatre octets décimaux séparés par des points : `192.168.56.20`. Chaque octet va de 0 à 255.

DÉFINITION Masque / préfixe
Indique quelle partie de l'adresse désigne le réseau et laquelle désigne l'hôte. Ainsi `192.168.56.0/24` signifie : les 24 premiers bits (`192.168.56`) identifient le réseau, les 8 derniers identifient l'hôte, soit 256 adresses.

Adresse particulière	Rôle
`192.168.56.0`	Adresse du réseau (non attribuée à un hôte)
`192.168.56.255`	Adresse de diffusion (broadcast) : tous les hôtes du réseau
`127.0.0.1`	Bouclage (loopback) : la machine elle-même
`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`	Plages privées (RFC 1918), non routées sur Internet

À RETENIR
L'adresse IP identifie un hôte **de bout en bout**, à travers plusieurs réseaux.

1.2 L'adresse MAC

DÉFINITION Adresse MAC
Identifiant d'une carte réseau sur le **lien local** (le même réseau physique ou Wi-Fi). 48 bits (6 octets), écrits en hexadécimal : `a8:86:dd:9b:cb:20`.

- Les 3 premiers octets (`a8:86:dd`) forment l'**OUI**, identifiant du constructeur. C'est pourquoi Wireshark affiche `Apple_9b:cb:20` : il remplace l'OUI par le nom du fabricant.
- Les 3 derniers octets (`9b:cb:20`) identifient la carte chez ce constructeur.
- L'adresse `ff:ff:ff:ff:ff:ff` est l'adresse MAC de **diffusion** : tous les équipements du lien la reçoivent.

PIÈGE
L'adresse MAC ne traverse pas un routeur. Quand votre machine parle à un serveur sur Internet, la MAC de destination dans la trame est celle de votre routeur (passerelle), pas celle du serveur. C'est exactement ce que l'on verra dans la capture du ping vers `172.217.19.131`.

1.3 Le modèle en couches et l'encapsulation

DÉFINITION Encapsulation
Une communication réseau est construite en couches. Chaque couche ajoute son en-tête devant les données reçues de la couche supérieure. À l'arrivée, chaque couche retire son en-tête : c'est la **désencapsulation**.

Couche TCP/IP	Unité de données	Adresse utilisée	Exemples de protocoles
Application	Message / données	URL, nom DNS	HTTP, DNS, SSH
Transport	Segment (TCP) / Datagramme (UDP)	Port	TCP, UDP
Internet	Paquet	Adresse IP	IPv4, IPv6, ICMP
Accès réseau	Trame	Adresse MAC	Ethernet, Wi-Fi

Chaque couche ajoute son en-tête devant les données de la couche supérieure

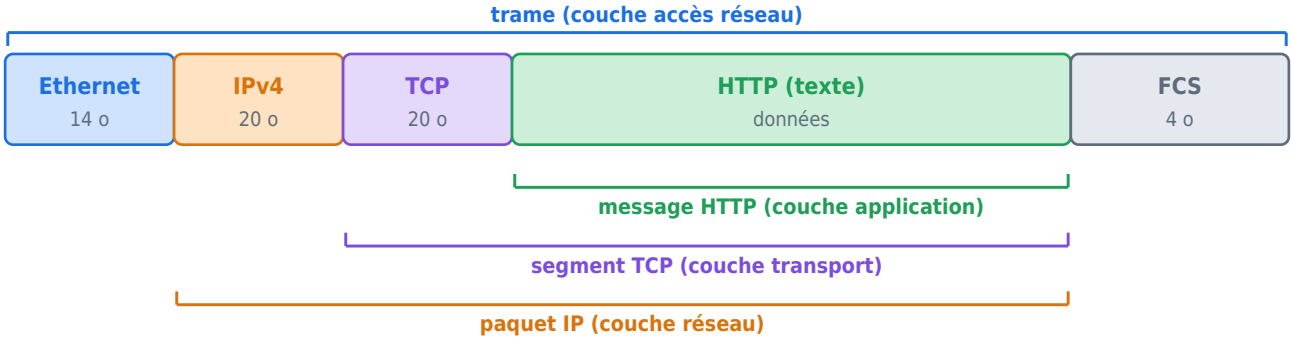


Schéma — encapsulation d'une requête web (de l'application vers le câble)

Wireshark fait le travail de désencapsulation à votre place et vous montre chaque couche dans un panneau dédié (voir partie 2).

À RETENIR
trame = Ethernet ; **paquet** = IP ; **segment** = TCP ; **message** = application. Dans le langage courant on dit « paquet » pour tout, mais en analyse la distinction est utile.

1.4 Port, service, application

DÉFINITION Port
Nombre de 0 à 65535, associé à TCP ou UDP, qui désigne un **service** sur une machine. L'adresse IP désigne la machine ; le port désigne le service.

Port	Protocole	Service habituel
22	TCP	SSH (administration à distance chiffrée)
53	UDP/TCP	DNS (résolution de noms)
80	TCP	HTTP (web non chiffré)
443	TCP	HTTPS (web chiffré par TLS)
3000	TCP	Port couramment utilisé par des applications Node.js ; ici Juice Shop
3389	TCP	RDP (bureau à distance Windows)

PIÈGE
Le numéro de port n'est qu'une **convention**. Un serveur web peut écouter sur 3000, un service SSH sur 2222. C'est pourquoi Nmap vérifie le service réel (option `-sV`) et ne se contente pas de la table ci-dessus. Pour le port 3000, Nmap peut afficher un nom approximatif (`ppp` ou `http` selon sa base) : l'observation à retenir est « un service répond sur le port 3000 ».

1.5 TCP, UDP et ICMP

- **TCP** est orienté connexion : il établit une session, numérote les données, retransmet en cas de perte. HTTP, HTTPS et SSH l'utilisent.
- **UDP** envoie des datagrammes sans connexion ni garantie. DNS, la découverte de services locaux et la voix l'utilisent.
- **ICMP** n'est pas un protocole de transport applicatif : c'est un protocole de contrôle et de diagnostic du réseau (`ping`, messages « destination injoignable »).

L'établissement d'une connexion TCP (poignée de main en trois temps)

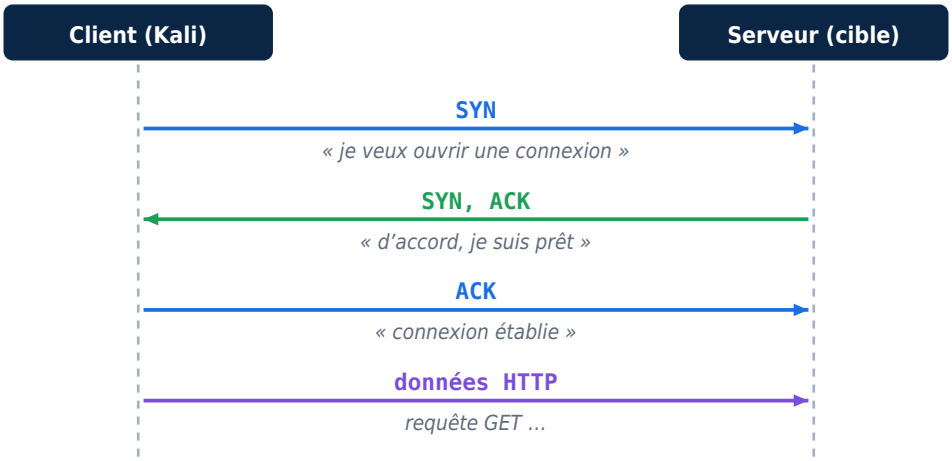


Schéma — la poignée de main TCP (three-way handshake)

À RETENIR Lien avec Nmap
Un port **ouvert** répond SYN/ACK ; un port **fermé** répond RST (refus) ; un port **filtré** ne répond rien ou renvoie une erreur ICMP, car un pare-feu intercepte le paquet.

1.6 Bannière, version, HTTP en clair

DÉFINITION Bannière
Texte qu'un service renvoie en se présentant (par exemple `SSH-2.0-OpenSSH_9.6`). Elle peut révéler le logiciel et sa version. Une version exposée n'est pas une faille en soi, mais elle aide un attaquant à chercher des vulnérabilités connues : c'est de l'**information technique exposée**.

HTTP est un protocole **textuel et non chiffré**. Une requête ressemble à ceci :

requête HTTP (texte brut lisible sur le réseau)

```
GET /robots.txt HTTP/1.1
Host: 192.168.56.20:3000
User-Agent: curl/8.5.0
Accept: */*
```

Toute personne qui observe le réseau entre le client et le serveur peut lire ce texte. HTTPS enveloppe ce contenu dans TLS : un observateur voit alors des octets illisibles.

RÈGLE R3 Chiffrer les échanges
Tout échange contenant une authentification ou des données personnelles doit être chiffré (HTTPS, SSH). Un protocole en clair (HTTP, FTP, Telnet) expose son contenu à quiconque peut observer le réseau.

RÈGLE R4 Réduire la surface d'exposition
Moins un système expose de services, de ports et d'informations (versions, bannières), moins il offre de prises à un attaquant. On ferme ce qui est inutile et on n'expose que le nécessaire.

1.7 Reconnaissance, recherche de vulnérabilités, exploitation

Nous restons en **cybersécurité défensive** : on cherche ce qu'un organisme expose, afin de le **réduire**.

Phase	Question posée	Exemples d'outils	Dans cet atelier
Reconnaissance	Qu'est-ce qui existe et répond ?	Nmap, Gobuster, ExifTool, Wireshark	**Oui**
Recherche de vulnérabilités	Ce qui répond est-il faible ?	Nikto, scanners de vulnérabilités	Oui, en nuançant les résultats
Exploitation	Peut-on utiliser la faiblesse ?	Outils d'exploitation	**Non**

1.8 Le vocabulaire de l'analyse

Cinq mots à ne jamais confondre :

Terme	Définition	Exemple dans l'atelier
Observation	Un fait relevé, sans interprétation	Le port 3000/tcp est ouvert
Preuve	Élément vérifiable qui soutient l'observation	Fichier `scan.txt`, capture `httptest.pcapng`
Faux positif	Alerte d'un outil qui s'avère inexacte	Gobuster signale `/backup` mais l'application renvoie sa page d'accueil
Exposition potentielle	Une donnée pourrait être accessible mais n'est pas démontrée	Un en-tête de sécurité est absent
Vulnérabilité confirmée	Faiblesse vérifiée (version, contexte, test autorisé)	Version vulnérable identifiée et applicable

DÉFINITION Risque
Combinaison de la **vraisemblance** (quelle chance que cela arrive ?) et de l'**impact** (quelles conséquences ?). Un port ouvert n'est pas un risque en soi ; un port ouvert exposant un service vulnérable, accessible à un attaquant, qui protège des données personnelles, en est un.

NE PAS CONCLURE
« Nikto a trouvé 20 alertes, donc le serveur est vulnérable ». Chaque alerte doit être vérifiée.

RÈGLE R5 Séparer faits, hypothèses et recommandations
Un rapport distingue ce qui est **démontré** (fait, avec preuve), ce qui est **supposé** (hypothèse à vérifier) et ce qui est **proposé** (recommandation). On ne présente jamais une hypothèse comme un fait.

RÈGLE R6 Toute alerte se vérifie
Un outil automatique produit des **indices**, pas des preuves. Avant de classer une alerte, on la vérifie par un second moyen (commande manuelle, lecture du contenu, comparaison avec la documentation).

1.9 Architecture du laboratoire

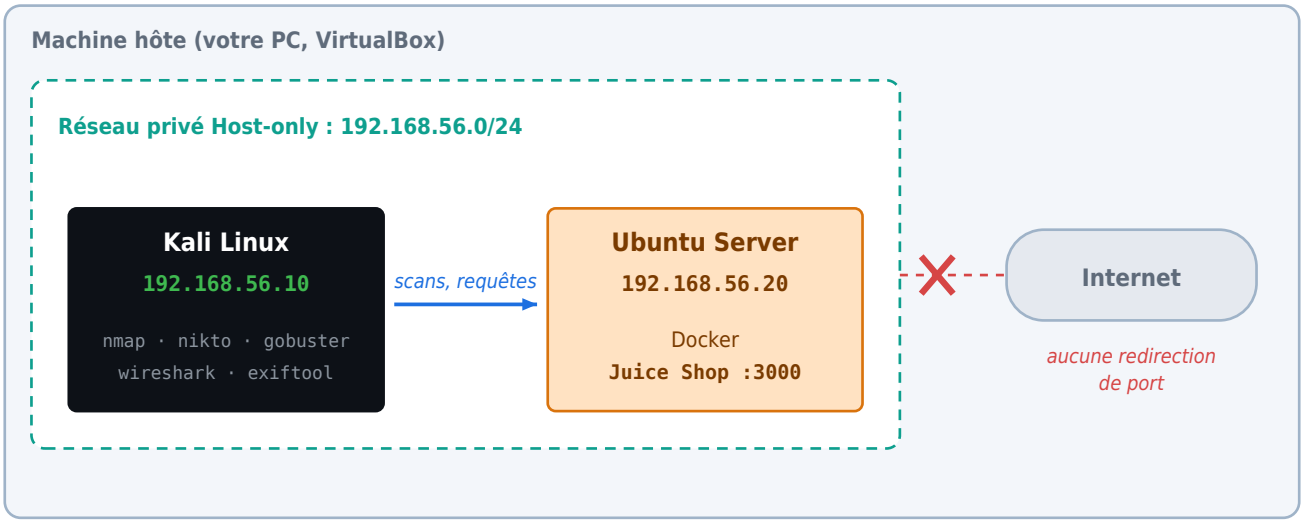


Schéma — architecture du laboratoire (adresses indicatives : adaptez-les et notez-les dans votre rapport)

PARTIE 2

60 min

Mode d'emploi : lire une trame dans Wireshark

OBJECTIF

Savoir lire une trame couche par couche, vérifier les champs dans les octets bruts et formuler une conclusion limitée à ce que l'observation démontre.

2.1 Le principe : lire de l'extérieur vers l'intérieur

Une trame est une succession d'enveloppes emboîtées, comme des **poupées russes**. On la lit de l'extérieur vers l'intérieur :

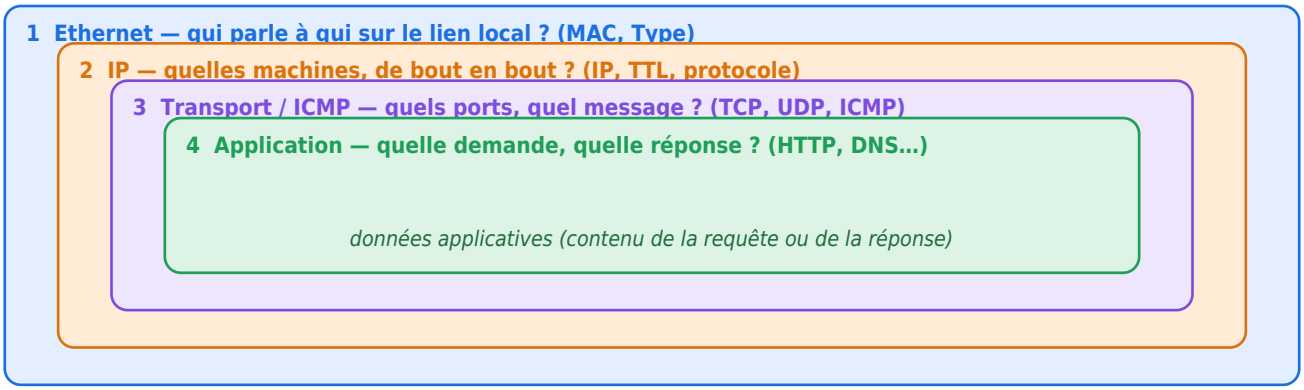


Schéma — ordre de lecture d'une trame

À chaque niveau, un champ indique quel protocole vient ensuite : le champ **Type** d'Ethernet (`0x0800` = IPv4), le champ **Protocol** d'IPv4 (`1` = ICMP, `6` = TCP, `17` = UDP). C'est ce chaînage que Wireshark utilise pour décoder.

2.2 Anatomie d'une trame Ethernet II

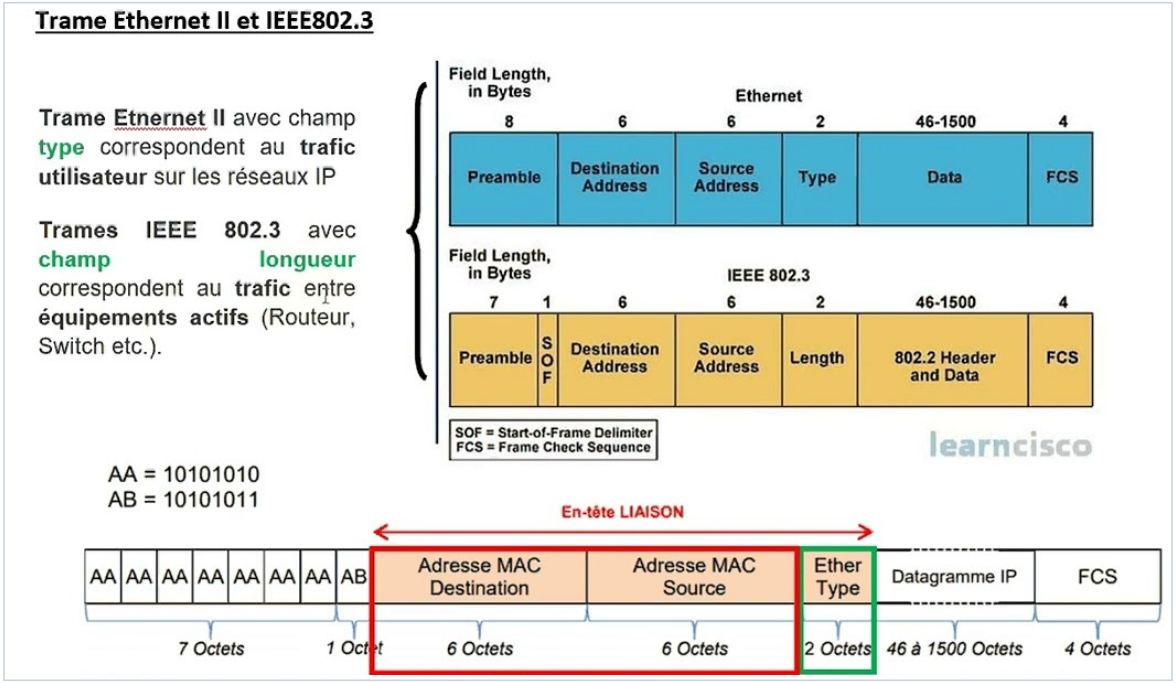


Figure 1 — Trame Ethernet II et IEEE 802.3 (schéma de cours)

Champ	Taille	Rôle
Préambule	7 octets	Synchronisation de l'horloge du récepteur (`AA AA AA AA AA AA AA`)
SFD (début de trame)	1 octet	Délimiteur `AB` : le contenu commence juste après
Adresse MAC destination	6 octets	Qui doit recevoir la trame sur le lien local
Adresse MAC source	6 octets	Qui l'a émise
Type (EtherType)	2 octets	Protocole transporté (voir tableau ci-dessous)
Données (charge utile)	46 à 1500 octets	Le paquet IP, ou autre
FCS	4 octets	Contrôle d'erreur (CRC) de la trame

EtherType	Protocole transporté
`0x0800`	IPv4
`0x0806`	ARP (résolution adresse IP → adresse MAC)
`0x86DD`	IPv6
`0x8100`	VLAN 802.1Q (étiquette de VLAN)

- Wireshark n'affiche généralement ni le préambule, ni le SFD, ni le FCS : la carte réseau les traite et les retire. Une trame visible comprend donc **14 octets d'en-tête** (6 + 6 + 2) plus la charge utile.
- La charge utile maximale est de 1500 octets (le **MTU** Ethernet) : une trame vue dans Wireshark fait au maximum **1514 octets** (14 + 1500).
- La taille minimale d'une trame sur le câble est de 64 octets avec FCS. Si les données sont trop courtes, l'émetteur ajoute du **bourrage** (padding) : on peut voir de petites trames complétées par des zéros.
- Ethernet II utilise le champ de 2 octets comme **Type** (valeur ≥ `0x0600`). IEEE 802.3 l'utilise comme **Longueur** (valeur ≤ 1500). Sur les réseaux IP on rencontre presque toujours Ethernet II ; le champ Longueur apparaît notamment dans les trames STP.

À RETENIR
Les 14 premiers octets d'une trame se lisent toujours de la même façon : 6 octets de destination, 6 octets de source, 2 octets de type.

2.3 Anatomie d'un paquet IPv4

La charge utile de la trame Ethernet est un paquet IPv4. Son en-tête fait **20 octets** sans options.

Octets	Champ	Rôle
0	Version (4 bits) + IHL (4 bits)	Version = 4 ; IHL = longueur de l'en-tête en mots de 4 octets (5 → 20 octets). D'où l'octet `45`
1	DSCP / ECN	Priorité de service (souvent `00`)
2–3	Longueur totale	En-tête + données IP, en octets
4–5	Identification	Identifiant du paquet (utile en fragmentation)
6–7	Drapeaux + décalage de fragment	Gestion de la fragmentation (`00 00` ou `40 00` = ne pas fragmenter)
8	TTL	Durée de vie : décrémentée à chaque routeur, le paquet est détruit à 0
9	Protocole	Protocole suivant : `01` ICMP, `06` TCP, `11` UDP (17 en décimal)
10–11	Somme de contrôle de l'en-tête	Détection d'erreurs dans l'en-tête
12–15	Adresse IP source	Qui a émis le paquet
16–19	Adresse IP destination	À qui il est destiné

DÉFINITION **TTL (Time To Live)**
Durée de vie d'un paquet, décrémentée à chaque routeur traversé. Valeurs initiales courantes : 64 (Linux, macOS), 128 (Windows), 255 (certains équipements réseau). Un TTL de 118 reçu en réponse suggère une valeur initiale de 128 diminuée de 10 sauts : c'est une **hypothèse raisonnable, pas une preuve** du système d'exploitation distant.

2.4 Les zones de l'interface Wireshark

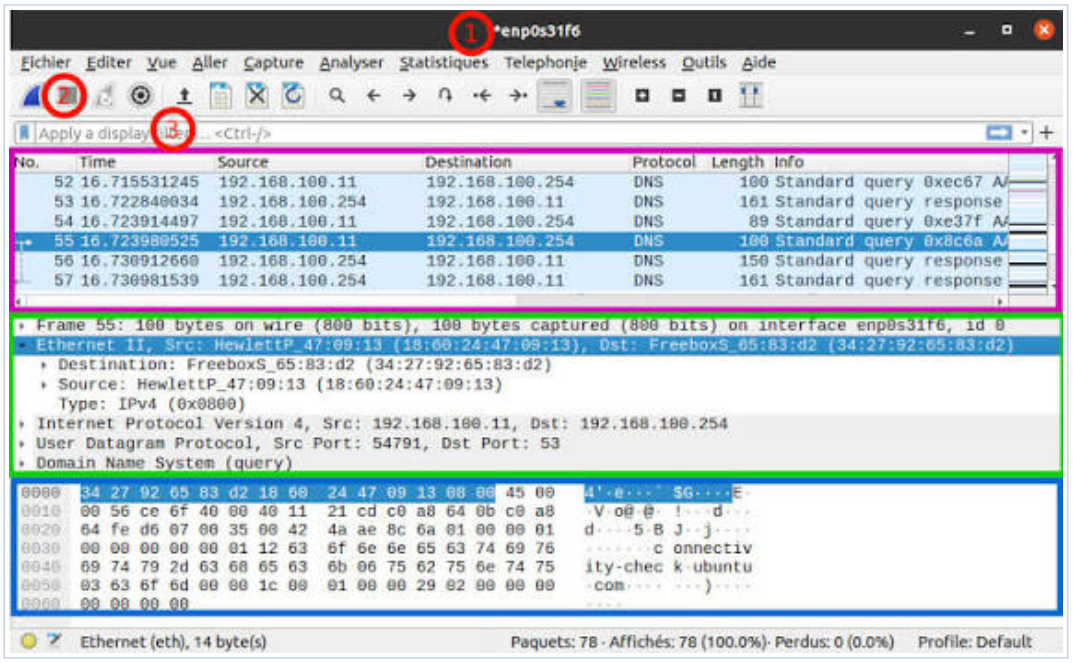


Figure 2 — Capture Wireshark en français, interface enp0s31f6 (zones colorées : ① liste en rose, ② détail en vert, ③ octets en bleu)

Zone	Rôle	Remarque
Barre de menus	Fichier, Éditer, Vue, Aller, Capture, Analyser, Statistiques...	**Capture → Options** pour choisir l'interface
Barre d'outils	Démarrer (aileron bleu), arrêter (carré rouge), redémarrer, ouvrir, enregistrer, rechercher	Le carré rouge n'est actif que pendant une capture
Barre de filtre d'affichage	Saisie d'un filtre qui masque les paquets non concernés	Fond **vert** : filtre valide ; **rouge** : erreur de syntaxe ; **jaune** : valide mais à vérifier
① Liste des paquets	Une ligne par paquet : No., Time, Source, Destination, Protocol, Length, Info	Vue d'ensemble : c'est ici que l'on sélectionne un paquet
② Détail des protocoles	Le paquet décomposé couche par couche (Frame, Ethernet II, IPv4, TCP/UDP/ICMP, application)	Chaque ligne se déplie avec la flèche ▸
③ Octets bruts	Contenu hexadécimal (gauche) et texte ASCII (droite)	Les octets d'un champ sélectionné en ② sont surlignés
Barre d'état	Nom du champ sélectionné, nombre de paquets capturés / affichés / perdus, profil	`Packets: 87 · Displayed: 8 (9,2%)` : le filtre a retenu 8 paquets sur 87

BONNE PRATIQUE **Liaison entre les zones**
Cliquez sur un champ dans la zone ② (par exemple Source dans Ethernet II) et les 6 octets correspondants se surlignent dans la zone ③ ; la barre d'état affiche « Source Hardware Address (eth.src), 6 bytes ». C'est la fonction la plus utile pour comprendre comment les champs sont codés.

Les couleurs

Wireshark colore les lignes selon des règles (menu **Vue → Règles de coloration**). Les couleurs **ne sont pas une preuve** : elles aident à repérer des familles de trafic (ICMP en rose pâle, TCP en violet clair, HTTP en vert, UDP en bleu pâle, erreurs en noir/rouge dans le profil par défaut).

Les colonnes de la liste des paquets

Colonne	Signification
No.	Numéro d'ordre dans la capture. Pas dans le paquet lui-même
Time	Secondes écoulées depuis le début de la capture (modifiable : Vue → Format d'affichage de l'heure)
Source / Destination	Adresses du niveau le plus élevé connu : IP, sinon MAC
Protocol	Protocole le plus haut reconnu dans la trame
Length	Taille de la trame capturée, en octets (14 octets d'en-tête Ethernet inclus)
Info	Résumé produit par Wireshark : requête, réponse, ports, numéros de séquence…

2.5 La méthode en 10 étapes

1. **Appliquer un filtre d'affichage** adapté (`icmp`, `dns`, `http`, `ip.addr == …`) pour réduire le bruit.
2. **Lire la liste des paquets** : repérer le sens (qui parle à qui), le protocole, la taille et le résumé.
3. **Sélectionner un paquet** et vérifier dans la barre d'état le nombre d'octets.
4. **Dérouler Frame** : numéro de trame, taille sur le fil, taille capturée, interface. Informations sur la capture, pas des champs de la trame.
5. **Dérouler Ethernet II** : MAC de destination, MAC de source, Type.
6. **Dérouler Internet Protocol** : IP source, IP destination, TTL, protocole suivant, longueur totale.
7. **Dérouler la couche transport** : TCP (ports, drapeaux, numéros de séquence), UDP (ports, longueur) ou ICMP (type, code).
8. **Dérouler la couche application** si elle existe (HTTP, DNS…).
9. **Vérifier dans les octets bruts** que les champs importants sont bien ceux annoncés (cliquer sur un champ et observer le surlignage).
10. **Formuler une conclusion** limitée à ce que l'observation démontre : distinguer faits, hypothèses et recommandations.

> **PIÈGE**
> Lire les couleurs ou la colonne Info comme une preuve. La colonne Info est un résumé calculé par Wireshark ; la vérité se trouve dans les champs des couches et dans les octets.

2.6 Cas n°1 — Lire un ping (ICMP), octet par octet

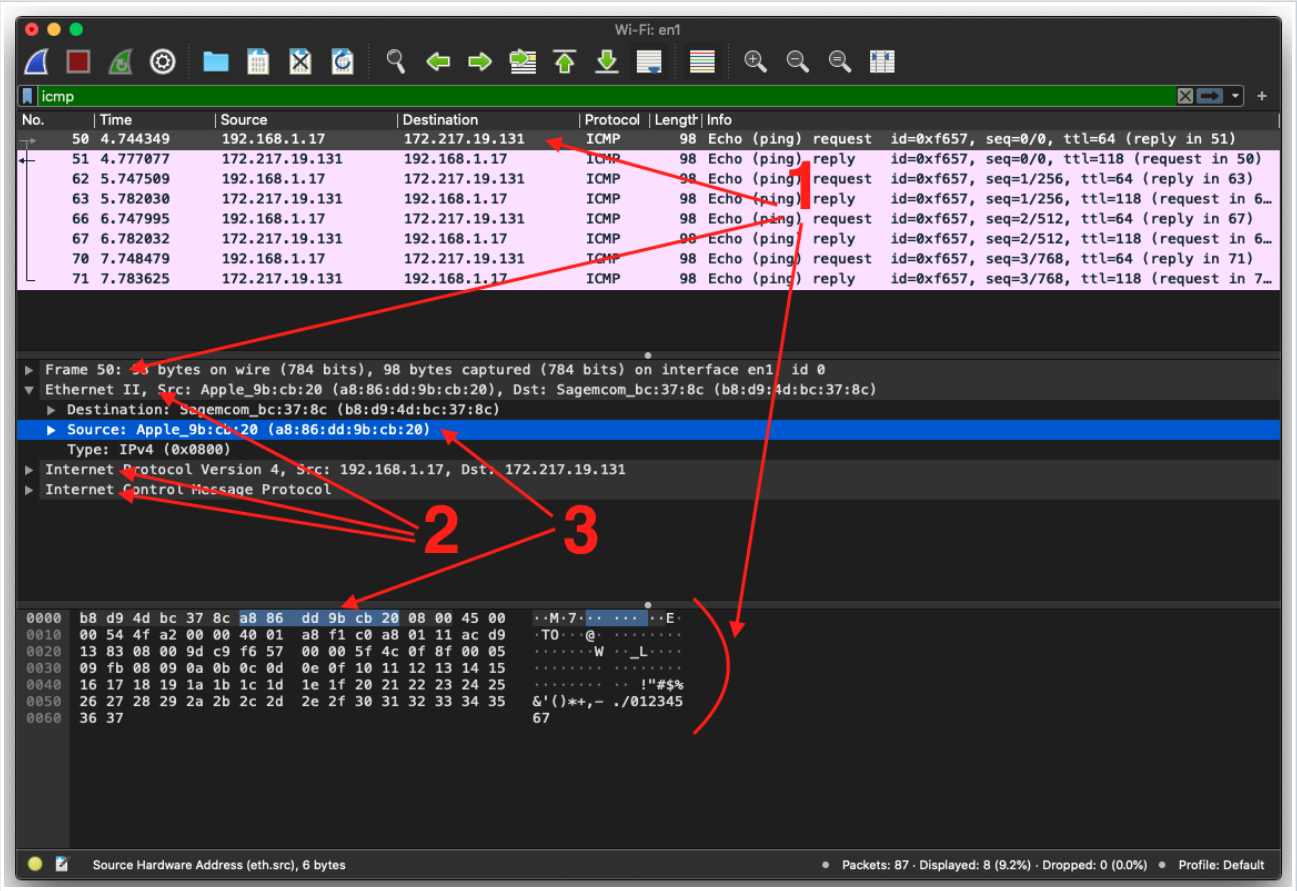


Figure 3 — Capture annotée du ping vers 172.217.19.131 : 1 liste filtrée par icmp, 2 détail des couches, 3 octets bruts

ÉTAPE A — La liste des paquets (zone 1)

Le filtre `icmp` affiche 8 paquets sur 87 capturés (barre d'état : `Displayed: 8 (9,2%)`).

filtre : icmp — Displayed: 8 (9,2 %)

No.	Time(s)	Source	Destination	Info
50	4,744349	192.168.1.17	172.217.19.131	Echo (ping) request id=0xf657 seq=0/0 ttl=64
51	4,777077	172.217.19.131	192.168.1.17	Echo (ping) reply id=0xf657 seq=0/0 ttl=118
62	5,747509	192.168.1.17	172.217.19.131	request seq=1/256 ttl=64
63	5,782030	172.217.19.131	192.168.1.17	reply seq=1/256 ttl=118

- Les requêtes sont envoyées environ **une par seconde** (4,74 s ; 5,74 s ; 6,74 s ; 7,74 s) : rythme par défaut de `ping`.
- Chaque requête a sa réponse : Wireshark les relie (`reply in 51`, `request in 50`).
- Le **délai aller-retour (RTT)** de la première paire se calcule par différence : 4,777077 − 4,744349 = 0,032728 s, soit environ **32,7 ms**.
- L'identifiant (`0xf657`) est identique pour tout le ping : il identifie la commande `ping` qui a émis ces requêtes. Le numéro de séquence augmente de 1 à chaque requête.
- Pourquoi `seq=1/256` ? Le champ occupe 2 octets (`00 01`). Wireshark l'affiche en gros-boutiste (big-endian : 1) puis en petit-boutiste (little-endian : `0x0100` = 256). Seule la première valeur est conventionnelle ici.
- Le TTL passe de 64 (requête) à 118 (réponse) : deux machines différentes émettent, avec des valeurs initiales différentes.

ÉTAPE B Le détail des couches (zone 2) pour le paquet 50

Ligne Wireshark	Information lue
`Frame 50: 98 bytes on wire (784 bits), 98 bytes captured … interface en1`	Taille 98 octets = 784 bits (98 × 8). Interface de capture : `en1` (Wi-Fi sur macOS)
`Ethernet II, Src: Apple_9b:cb:20 (a8:86:dd:9b:cb:20), Dst: Sagemcom_bc:37:8c (b8:d9:4d:bc:37:8c)`	Source : un Mac. Destination : la box (Sagemcom), et **non** Google
`Type: IPv4 (0x0800)`	La charge utile est un paquet IPv4
`Internet Protocol Version 4, Src: 192.168.1.17, Dst: 172.217.19.131`	Machine locale → serveur distant
`Internet Control Message Protocol`	Type 8 (Echo request), code 0

À RETENIR **Pourquoi la MAC de destination est-elle celle de la box ?**
Parce que `172.217.19.131` n'est pas sur le réseau local. La trame va jusqu'au routeur, qui la reconstruit pour le saut suivant. L'adresse IP reste, elle, celle du serveur final.

ÉTAPE C Les octets bruts (zone 3) : décodage complet

La colonne de gauche (`0000`, `0010`…) est le **décalage (offset)** en hexadécimal du premier octet de la ligne ; chaque ligne contient 16 octets. Les champs sont colorés ci-dessous selon la légende.

dump hexadécimal — Frame 50 (98 octets)

```
0000  b8 d9 4d bc 37 8c a8 86 dd 9b cb 20 08 00 45 00
0010  00 54 4f a2 00 00 40 01 a8 f1 c0 a8 01 11 ac d9
0020  13 83 08 00 9d c9 f6 57 00 00 5f 4c 0f 8f 00 05
0030  09 fb 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15
0040  16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25
0050  26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35
0060  36 37

MAC destination  MAC source  EtherType IPv4  en-tête IPv4  TTL / type ICMP  protocole
IP source  IP destination  identifiant ICMP  n° de séquence
```

Décalage	Octets	Champ	Valeur décodée
0x00–0x05	`b8 d9 4d bc 37 8c`	MAC destination	`b8:d9:4d:bc:37:8c` (la box)
0x06–0x0b	`a8 86 dd 9b cb 20`	MAC source	`a8:86:dd:9b:cb:20` (le Mac)
0x0c–0x0d	`08 00`	Type Ethernet	IPv4
0x0e	`45`	Version + IHL	Version 4 ; IHL 5 → en-tête de 5 × 4 = 20 octets
0x0f	`00`	DSCP/ECN	Pas de classe de service particulière
0x10–0x11	`00 54`	Longueur totale IP	0x0054 = 84 octets
0x12–0x13	`4f a2`	Identification IP	0x4fa2
0x14–0x15	`00 00`	Drapeaux / fragment	Non fragmenté
0x16	`40`	TTL	0x40 = 64
0x17	`01`	Protocole	1 = ICMP
0x18–0x19	`a8 f1`	Somme de contrôle IP	Contrôle de l'en-tête
0x1a–0x1d	`c0 a8 01 11`	IP source	192.168.1.17
0x1e–0x21	`ac d9 13 83`	IP destination	172.217.19.131
0x22	`08`	Type ICMP	8 = Echo request
0x23	`00`	Code ICMP	0

Décalage	Octets	Champ	Valeur décodée
0x24–0x25	`9d c9`	Somme de contrôle ICMP	Contrôle du message ICMP
0x26–0x27	`f6 57`	Identifiant	0xf657
0x28–0x29	`00 00`	Numéro de séquence	0
0x2a–0x31	`5f 4c 0f 8f 00 05 09 fb`	Horodatage	8 octets (valeur dépendant de l'émetteur)
0x32–0x61	`08 09 … 36 37`	Données de remplissage	48 octets : suite croissante 0x08 à 0x37

BONNE PRATIQUE **Vérification arithmétique**

L'en-tête ICMP fait 8 octets, la charge utile 56 (8 + 48), soit 64 octets ICMP. En ajoutant l'en-tête IP de 20 octets : **84 octets**, la longueur totale IP `00 54`. En ajoutant les 14 octets Ethernet : **98 octets**, soit la valeur de `Frame 50`. Tout est cohérent.

À RETENIR **Lire une adresse IP en hexadécimal**

`c0` = 12 × 16 + 0 = 192 ; `a8` = 10 × 16 + 8 = 168 ; `01` = 1 ; `11` = 1 × 16 + 1 = 17 → **192.168.1.17**. De même `ac d9 13 83` donne **172.217.19.131**.

La colonne ASCII (à droite) n'est pas un second contenu : c'est **la même suite d'octets**, rendue en caractères lorsque c'est possible, avec un point pour les octets non imprimables. Ici les octets `0x20` à `0x37` s'affichent `!"#$%&'()*+,-./01234567` : c'est le remplissage standard du ping, il ne contient pas de message.

ÉTAPE D **Conclusion**

ATTENDU **Ce que la capture démontre**

L'hôte `192.168.1.17` a envoyé des requêtes ICMP Echo à `172.217.19.131` et a reçu des réponses ; le délai aller-retour est d'environ 33 ms.

NE PAS CONCLURE

Que le serveur web de cette adresse fonctionne (le ping ne teste que l'ICMP), que l'hôte distant est vulnérable, ou qu'il tourne sous tel système d'exploitation (le TTL ne donne qu'une indication).

2.7 Cas n°2 — Diffusions et trames inconnues

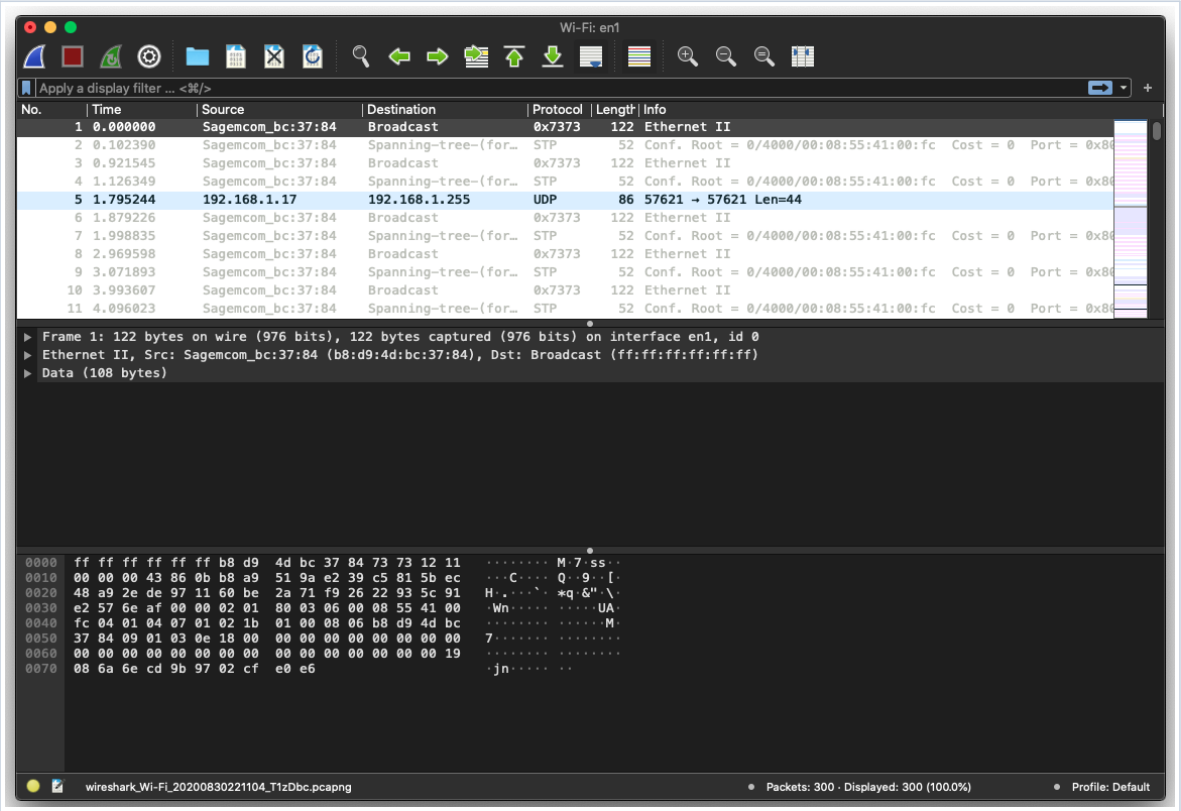


Figure 4 — Capture Wi-Fi de 300 paquets non filtrés (trames STP, diffusions, ligne UDP en surbrillance)

Cette capture n'a **pas de filtre** : les paquets de toutes sortes se mélangent. C'est la situation réelle à laquelle vous serez confrontés.

La première ligne. Paquet 1, 122 octets : Source `Sagemcom_bc:37:84`, Destination `Broadcast`, colonne Protocol `0x7373`, Info `Ethernet II`. Dans le détail, Wireshark n'affiche que trois lignes : `Frame`, `Ethernet II` et `Data (108 bytes)`.

- La destination `ff:ff:ff:ff:ff:ff` (visible au début des octets) est l'adresse de **diffusion**.
- Le champ Type vaut `0x7373`. Ce n'est pas un EtherType courant : Wireshark n'a pas de décodeur pour lui, il affiche donc le reste en « Data ». 122 − 14 = 108 octets de données : le calcul est exact.
- La source est un équipement Sagemcom (la box) ; son adresse `…:37:84` est voisine de celle vue sur la capture du ping (`…:37:8c`), ce qui suggère une autre interface de la même box. **C'est une hypothèse.**

> **NE PAS CONCLURE**
> Qu'une trame d'un type inconnu est une attaque. De nombreux équipements réseau émettent des trames propriétaires de gestion ou de découverte. L'observation correcte est : « trame de diffusion de type non décodé émise par la box ».

Les trames STP (52 octets, destination `Spanning-tree-(for-bridges)`). STP (Spanning Tree Protocol) sert aux commutateurs à éviter les boucles réseau. Info : `Conf. Root = 0/4000/00:08:55:41:00:fc Cost = 0 Port = 0x80…`. Ces trames utilisent le champ **Longueur** d'IEEE 802.3 et non un EtherType, ce qui illustre la figure 1.

Le paquet 5 (surbrillance bleue) : `192.168.1.17 → 192.168.1.255`, UDP, `57621 → 57621 Len=44`, 86 octets.

- Adresse IP destination `.255` : **diffusion IP** du réseau `192.168.1.0/24`.
- Vérification de la taille : 14 (Ethernet) + 20 (IPv4) + 8 (UDP) + 44 (données) = **86**.
- Port UDP 57621 : souvent associé à la découverte d'appareils sur le réseau local par certaines applications de streaming musical. C'est une **indication, pas une preuve** : seule l'analyse du contenu ou du processus émetteur permettrait de l'affirmer.

2.8 Cas n°3 — Lire une requête DNS (UDP)

Sur la figure 2 (capture en français, interface `enp0s31f6`), on lit :

Zone	Contenu
Liste	Paquets 53 à 57 : `Standard query` et `Standard query response` entre `192.168.100.11` et `192.168.100.254`
Frame 55	100 octets (800 bits)
Ethernet II	Source `HewlettP_47:09:13` (18:60:24:47:09:13), destination `FreeboxS_65:83:d2` (34:27:92:65:83:d2), type IPv4 `0x0800`
IPv4	`192.168.100.11 → 192.168.100.254`
UDP	Port source 54791, port destination 53
DNS (query)	Demande de résolution

- Le **port destination 53** identifie le service DNS. Le port source 54791 est un **port éphémère** choisi par le système client pour cette requête ; il change à chaque demande.
- Dans les octets, `34 27 92 65 83 d2 | 18 60 24 47 09 13 | 08 00 | 45 00` : même structure que pour le ping (MAC destination, MAC source, EtherType IPv4, début de l'en-tête IP).
- La colonne ASCII laisse lire `ity-check.ubuntu.com`. Dans un message DNS, chaque étiquette du nom est précédée de sa longueur ; les points affichés remplacent ces octets de longueur. Le nom complet est `connectivity-check.ubuntu.com`, requête habituelle de vérification de connectivité.
- Vérification de la taille : 100 − 14 − 20 − 8 = 58 octets DNS. En-tête DNS 12 + nom 31 + type et classe 4 = 47 ; les 11 octets restants correspondent typiquement à un enregistrement d'option EDNS0. **Hypothèse cohérente avec le calcul.**
- Le serveur DNS est ici la passerelle (`.254`), qui est aussi le destinataire de la trame : l'IP et la MAC de destination désignent le **même équipement**, car il est sur le réseau local.

À RETENIR **Comparer avec le ping**
Pour 172.217.19.131, la MAC de destination est celle de la box mais l'IP est distante ; pour le DNS local, MAC et IP désignent le même appareil.

NE PAS CONCLURE
Que l'utilisateur de la machine a « visité » un site parce qu'un nom a été résolu. Beaucoup de résolutions sont émises automatiquement par le système.

2.9 Cas n°4 — Des encapsulations plus profondes

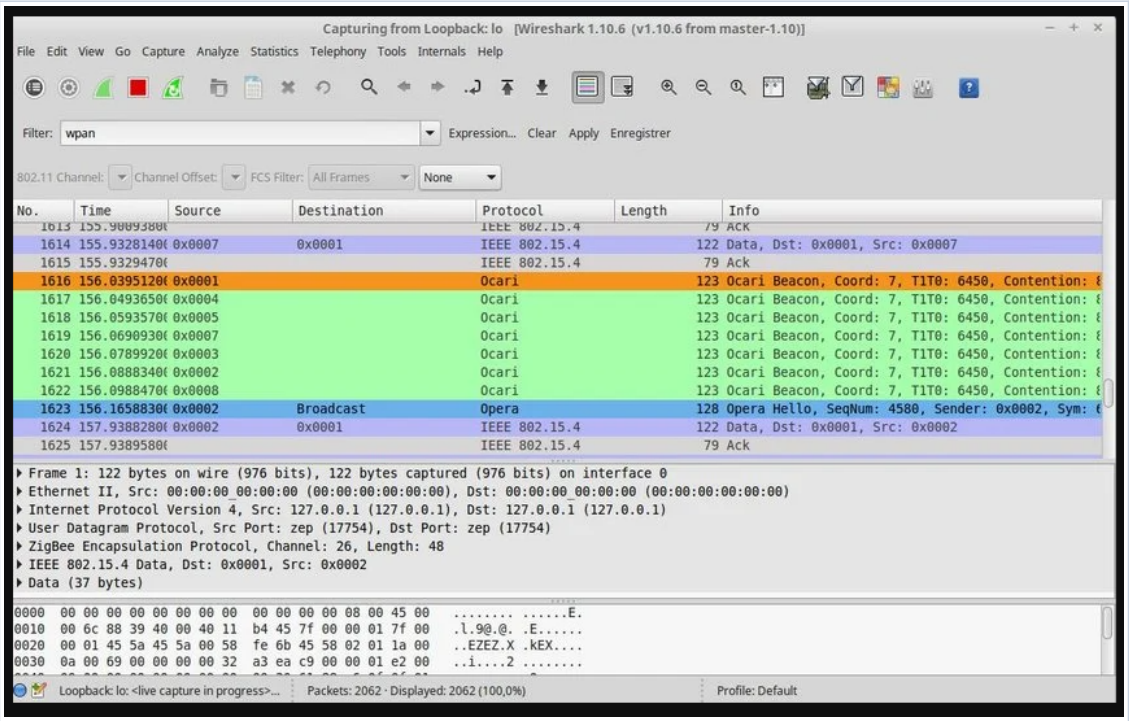


Figure 5 — Capture sur l'interface de bouclage lo avec le filtre wpan (Wireshark 1.10.6)

Cette capture montre que l'empilement des couches n'est pas toujours Ethernet → IP → TCP :

empilement des couches — Frame 1

```
Frame 1 : 122 octets (interface 0, "lo")
 └─ Ethernet II : 00:00:00:00:00:00 → 00:00:00:00:00:00
     └─ IPv4 : 127.0.0.1 → 127.0.0.1
         └─ UDP : port 17754 → port 17754 (ZEP)
             └─ ZigBee Encapsulation Protocol : canal 26, longueur 48
                 └─ IEEE 802.15.4 Data : 0x0002 → 0x0001
                     └─ Data : 37 octets
```

- Sur l'interface de bouclage `lo`, les adresses MAC sont toutes à zéro et les adresses IP sont `127.0.0.1` : le trafic **ne quitte pas la machine**.
- Le protocole **ZEP** encapsule des trames de réseau de capteurs radio (IEEE 802.15.4) dans de l'UDP. Un simulateur peut ainsi envoyer des trames radio via le réseau local.
- La colonne Protocol montre le protocole le plus profond reconnu (`Ocari`, `Opera`, `IEEE 802.15.4`). Dans les octets, les 12 premiers octets `00…` sont les deux MAC, puis `08 00` annonce IPv4 et `45 00` ouvre l'en-tête IP.

> **À RETENIR**
> La méthode reste la même, quelle que soit la profondeur de l'empilement : on descend couche par couche en suivant le champ qui annonce le protocole suivant.

2.10 Cas n°5 — La diffusion expliquée par un schéma

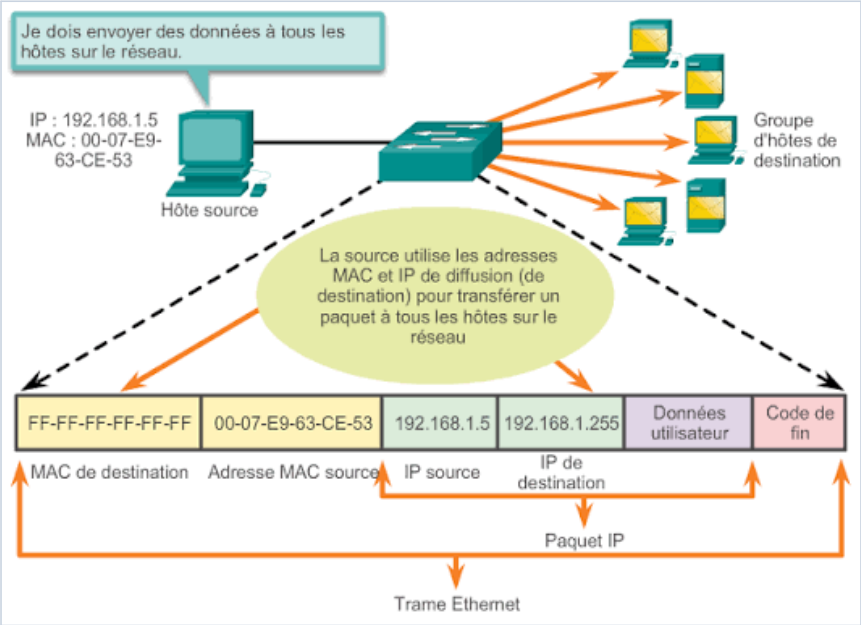


Figure 6 — Diffusion : hôte source 192.168.1.5 (MAC 00-07-E9-63-CE-53) émettant vers tous les hôtes du réseau

Zone de la trame	Valeur du schéma
MAC destination	`FF-FF-FF-FF-FF-FF`
MAC source	`00-07-E9-63-CE-53`
IP source	`192.168.1.5`
IP destination	`192.168.1.255`
Données utilisateur	Contenu applicatif
Code de fin	Contrôle d'erreur (FCS)

Le paquet IP (IP source, IP destination, données) est enfermé dans la trame Ethernet (MAC destination, MAC source, paquet IP, FCS). Le commutateur, recevant une trame dont la MAC destination est `FF-FF-FF-FF-FF-FF`, la copie vers **tous ses ports** (sauf celui d'entrée). C'est exactement ce que l'on a vu avec le paquet 5 de la figure 4.

2.11 Convertir : hexadécimal, décimal, binaire

Hexa	Décimal	Binaire
`0a`	10	00001010
`11`	17	00010001
`40`	64	01000000
`a8`	168	10101000
`c0`	192	11000000
`ff`	255	11111111

Méthode : un octet = deux chiffres hexadécimaux. Premier chiffre × 16 + second chiffre, avec a = 10, b = 11, c = 12, d = 13, e = 14, f = 15.

- `0x0054` = 0 × 4096 + 0 × 256 + 5 × 16 + 4 = **84**.
- `0x0800` = 8 × 256 = **2048** (EtherType IPv4).
- `0x0600` = 6 × 256 = **1536** : seuil entre Longueur (≤ 1500) et Type (≥ 1536).

2.12 Les filtres Wireshark

Deux familles à ne pas confondre

Type	Où se saisit-il ?	Effet	Syntaxe
Filtre de capture	Avant de démarrer (Capture → Options)	N'enregistre que les paquets correspondants	BPF : `host 192.168.56.20`, `port 3000`
Filtre d'affichage	Barre de filtre	Masque les autres paquets sans les supprimer	Wireshark : `ip.addr == 192.168.56.20`

Objectif	Filtre d'affichage
Voir uniquement le ping	`icmp`
Requêtes ping seulement	`icmp.type == 8`
Tout trafic d'une machine	`ip.addr == 192.168.1.17`
Trafic d'une adresse MAC	`eth.addr == a8:86:dd:9b:cb:20`
Diffusions Ethernet	`eth.dst == ff:ff:ff:ff:ff:ff`
Trafic vers le port 3000	`tcp.port == 3000`
Requêtes DNS	`dns` ou `udp.port == 53`
Requêtes HTTP GET	`http.request.method == "GET"`
Début de connexions TCP	`tcp.flags.syn == 1 && tcp.flags.ack == 0`
Connexions refusées	`tcp.flags.reset == 1`
Gros paquets	`frame.len > 1000`
Exclure l'ARP	`!arp`

Opérateurs : `==` égal, `!=` différent, `&&` et, `||` ou, `!` non. Le fond de la barre devient vert si la syntaxe est valide, rouge sinon.

2.13 Les erreurs d'interprétation les plus fréquentes

1. **Confondre MAC et IP** : la MAC change à chaque saut, l'IP reste celle de bout en bout.
2. **Croire que la taille Wireshark inclut le FCS** : en général non.
3. **Prendre la colonne Info pour une preuve** : elle résume ; les champs et octets font foi.
4. **Conclure sur le système distant d'après le TTL** : indication faible.
5. **Prendre un protocole inconnu pour une attaque** : « inconnu de Wireshark » ne signifie pas « malveillant ».
6. **Oublier le filtre actif** : la barre d'état indique combien de paquets sont masqués.
7. **Capturer sur la mauvaise interface** : aucun paquet attendu n'apparaît.
8. **Confondre filtre de capture et d'affichage** : un mauvais filtre de capture perd définitivement des paquets.
9. **Lire du texte dans l'ASCII et y voir un secret** : des octets de remplissage peuvent être imprimables.
10. **Déduire une compromission d'un simple échange** : un ping, une diffusion ou une requête DNS ne démontrent pas une faille.

2.14 Fiche de lecture d'une trame (à recopier)

Champ à renseigner	Valeur relevée
Fichier de capture, interface, filtre actif	
N° du paquet, heure relative	
Taille (Frame) et vérification arithmétique	
MAC source / destination (et fabricants)	
EtherType	
IP source / destination, TTL, protocole	
Transport : ports (TCP/UDP) ou type/code (ICMP)	
Application : requête ou réponse	
Octets vérifiés (décalages)	
Fait démontré	
Hypothèse (à vérifier)	
Ce que l'on ne peut pas conclure	

2.15 Exercices de lecture de trames

Exercice 1. Dans le dump du ping, que désignent les octets `ac d9 13 83` ?

Exercice 2. La longueur totale IP est `00 54`. Que vaut-elle en décimal, et comment retrouver les 98 octets du Frame ?

Exercice 3. Paquet 5 de la figure 4 : vérifier la taille de 86 octets et expliquer pourquoi la MAC destination est `ff:ff:ff:ff:ff:ff`.

Exercice 4. Paquet 55 de la figure 2 : quel est le port du serveur, quel est celui du client, et que désigne `connectivity-check.ubuntu.com` ?

Exercice 5. Calculer le délai aller-retour entre les paquets 62 et 63 (figure 3).

Exercice 6. Un collègue affirme : « le TTL de 118 prouve que le serveur distant tourne sous Windows et qu'il est à 10 sauts. » Corrigez cette phrase.

Exercice 7. Un étudiant rédige : « des trames de type `0x7373` sont présentes, le réseau est attaqué. » Rédigez une observation correcte.

Corrigés

Corrigé 1. C'est l'adresse IP destination : 0xac = 172, 0xd9 = 217, 0x13 = 19, 0x83 = 131, soit `172.217.19.131`.

Corrigé 2. 0x0054 = 5 × 16 + 4 = 84 octets (en-tête IP de 20 + message ICMP de 64). En ajoutant les 14 octets d'en-tête Ethernet : 84 + 14 = 98, soit la taille de `Frame 50`.

Corrigé 3. 14 + 20 + 8 + 44 = 86. La MAC destination est `ff:ff:ff:ff:ff:ff` parce que l'IP destination est `192.168.1.255` : un paquet de diffusion IP est transporté dans une trame de diffusion Ethernet pour que tous les hôtes du réseau la reçoivent.

Corrigé 4. Port du serveur : 53 (DNS). Port du client : 54791 (éphémère). Le nom demandé est celui d'un service de vérification de connectivité utilisé par Ubuntu ; la requête démontre qu'une résolution de nom a été demandée, pas qu'un utilisateur a navigué.

Corrigé 5. 5,782030 − 5,747509 = 0,034521 s, soit environ **34,5 ms**.

Corrigé 6. « Le TTL de 118 est compatible avec une valeur initiale de 128 diminuée de 10 sauts, ce qui est courant pour certains systèmes, mais ne prouve ni le système d'exploitation ni la distance réelle (les routeurs peuvent ne pas décrémenter comme prévu, et la valeur initiale est configurable). »

Corrigé 7. « Des trames de diffusion de type Ethernet `0x7373`, non décodées par Wireshark, émises par `Sagemcom_bc:37:84`, sont présentes dans la capture. Leur nature n'est pas déterminée. Aucun élément ne permet de conclure à une attaque ; vérifier la documentation de l'équipement ou comparer avec une capture de référence. »

PARTIE 3

Installer le laboratoire

45 min

3

OBJECTIF
Disposer de deux machines virtuelles qui communiquent sur un réseau privé, avec une application cible qui répond sur le port 3000. Chaque étape se termine par une **vérification** : ne passez pas à la suivante tant qu'elle échoue.

3.1 Prérequis matériels et logiciels

Élément	Recommandation
Mémoire de la machine hôte	8 Go minimum, 16 Go confortable (Kali 4 Go + Ubuntu 2 Go + système hôte)
Espace disque libre	60 Go environ (30 + 20 + marge)
Virtualisation matérielle	Activée dans le BIOS/UEFI (Intel VT-x ou AMD-V)
Logiciels	VirtualBox (version récente), image officielle Kali pour VirtualBox, ISO Ubuntu Server LTS

PIÈGE
Si VirtualBox refuse de démarrer une VM 64 bits, la virtualisation est désactivée dans le BIOS ou un autre hyperviseur l'occupe (Hyper-V sous Windows).

3.2 Choisir le mode réseau

VirtualBox propose plusieurs modes. Le choix détermine ce que vos VM peuvent atteindre.

Mode	VM ↔ VM	VM → Internet	Hôte ↔ VM	Usage dans l'atelier
NAT	Non	Oui	Non (sans redirection)	Temporaire : mises à jour
Host-only (réseau hôte uniquement)	Oui	Non	Oui	**Recommandé pour le labo**
Réseau interne	Oui	Non	Non	Isolement maximal
Pont (bridge)	Oui	Oui	Oui	**À éviter** : expose la cible au réseau réel

RÈGLE R8 Isoler le laboratoire
Une cible volontairement vulnérable ne doit **jamais** être exposée à un réseau réel. On utilise le mode Host-only, sans redirection de port, et on désactive la carte NAT une fois les installations terminées.

Configuration conseillée : chaque VM a deux cartes. La carte 1 en **NAT** sert aux installations ; la carte 2 en **Host-only** porte le trafic d'atelier. Une fois les logiciels installés, on désactive la carte NAT.

ÉTAPE 3.2.1 Créer le réseau Host-only

1. VirtualBox → **Outils** (ou Fichier) → **Gestionnaire de réseau hôte** → créer un réseau (souvent nommé `vboxnet0`). Notez son adresse : en général `192.168.56.1/24`.
2. Pour chaque VM (arrêtée) : **Configuration → Réseau → Adaptateur 1 : NAT** ; **Adaptateur 2 : Réseau privé hôte (Host-only)** → sélectionner `vboxnet0`.
3. **Ne créez aucune redirection de port.**

ATTENDU **Vérification**
L'adresse du réseau hôte correspond à la plage choisie ; les deux VM utilisent le même réseau Host-only.

3.3 Installer la machine Kali Linux

ÉTAPE 3.3.1 Importer l'image officielle

1. Téléchargez l'image officielle Kali pour VirtualBox sur le site de Kali.
2. Vérifiez l'empreinte de contrôle fournie.
3. Importez-la : **Fichier → Importer un appareil virtuel**.
4. Réglez **2 vCPU** et **4 Go de RAM**.
5. Démarrez. Les identifiants par défaut sont indiqués dans la documentation officielle de l'image ; **changez le mot de passe dès la première connexion**.

RÈGLE R10 **Vérifier l'intégrité de ce que l'on télécharge**
On télécharge uniquement depuis le site officiel et on compare l'empreinte (hash) publiée avec celle du fichier obtenu. Un fichier modifié peut contenir un code malveillant.

kali@kali: ~

```
kali@kali:~$ passwd
Changing password for kali.
Current password:
New password:
Retype new password:
passwd: password updated successfully
```

RÈGLE R13 **Changer les secrets par défaut**
Un mot de passe par défaut est connu de tous. Il se change dès la première connexion — y compris sur une machine de laboratoire, pour prendre le bon réflexe.

ÉTAPE 3.3.2 Mettre à jour et installer les outils (carte NAT active)

kali@kali: ~

```
kali@kali:~$ sudo apt update
kali@kali:~$ sudo apt full-upgrade -y
kali@kali:~$ sudo apt install -y nmap nikto gobuster wireshark tshark exiftool curl
```

Commande	Rôle
`sudo`	Exécute la commande avec les droits administrateur
`apt update`	Télécharge la liste des paquets disponibles (n'installe rien)
`apt full-upgrade -y`	Met à jour tous les paquets ; `-y` répond « oui » aux confirmations
`apt install -y …`	Installe les outils listés

ATTENDU
La dernière ligne n'affiche pas d'erreur `E:`. Selon la version, certains paquets sont déjà présents ; le nom d'un paquet peut varier.

ÉTAPE 3.3.3 Autoriser la capture sans droits excessifs

Wireshark capture via un composant privilégié (`dumpcap`). Pendant l'installation, répondez **« Oui »** à la question sur l'autorisation des utilisateurs non-root, puis ajoutez votre utilisateur au groupe `wireshark` :

kali@kali: ~

```
kali@kali:~$ sudo usermod -aG wireshark kali
# puis : déconnexion / reconnexion pour que le groupe soit pris en compte
kali@kali:~$ id | grep wireshark
uid=1000(kali) gid=1000(kali) groups=1000(kali),...,wireshark
```

RÈGLE R7 Moindre privilège
On accorde à un compte ou à un programme **uniquement** les droits nécessaires à sa tâche. Ici : autoriser la capture via le groupe `wireshark` plutôt que lancer toute l'interface graphique en administrateur.

ÉTAPE 3.3.4 Vérifier les installations

kali@kali: ~

```
kali@kali:~$ nmap --version
kali@kali:~$ nikto -Version
kali@kali:~$ gobuster version
kali@kali:~$ exiftool -ver
kali@kali:~$ tshark --version
```

ATTENDU
Chaque commande affiche un numéro de version. Une erreur `command not found` signale un paquet manquant.

3.4 Préparer la cible Ubuntu Server

Installez Ubuntu Server LTS en suivant l'assistant : langue, clavier, stockage guidé, nom d'hôte `cible`, utilisateur. Activez le serveur **OpenSSH** si l'assistant le propose (le port 22 apparaîtra dans Nmap).

ÉTAPE 3.4.1 Mettre à jour et lire la configuration réseau

ubuntu@cible: ~

```
ubuntu@cible:~$ sudo apt update
ubuntu@cible:~$ sudo apt upgrade -y
ubuntu@cible:~$ sudo apt install -y curl
ubuntu@cible:~$ ip -br addr
lo               UNKNOWN   127.0.0.1/8
enp0s3           UP        10.0.2.15/24
enp0s8           UP        192.168.56.20/24
```

Ligne	Lecture
`lo`	Bouclage
`enp0s3`	Carte NAT (adresse `10.0.2.15` attribuée par VirtualBox)
`enp0s8`	Carte Host-only. Si elle n'a pas d'adresse IPv4, il faut la configurer

À RETENIR
Les noms d'interface (`enp0s3`, `enp0s8`, `eth0`...) varient selon la machine. Repérez les vôtres avec `ip -br addr`.

ÉTAPE 3.4.2 Fixer l'adresse de la carte Host-only (Netplan)

Ubuntu Server utilise **Netplan**. Listez le fichier de configuration puis éditez-le avec `sudo nano`. Les espaces d'indentation sont **obligatoires** (YAML) ; pas de tabulations.

ubuntu@cible: ~

```
ubuntu@cible:~$ ls /etc/netplan/
ubuntu@cible:~$ sudo nano /etc/netplan/50-cloud-init.yaml
```

/etc/netplan/50-cloud-init.yaml (le nom du fichier peut varier)

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 192.168.56.20/24
```

ubuntu@cible: ~

```
ubuntu@cible:~$ sudo netplan apply
ubuntu@cible:~$ ip -br addr show enp0s8
enp0s8           UP         192.168.56.20/24
```

ATTENDU

`enp0s8 UP 192.168.56.20/24`. Si Netplan refuse la configuration, `sudo netplan try` indique la ligne fautive.

ÉTAPE 3.4.3 Installer Docker et lancer Juice Shop

Suivez la documentation officielle de Docker pour Ubuntu (lien en partie 8). Puis :

ubuntu@cible: ~

```
ubuntu@cible:~$ sudo docker run -d --name juice-shop --restart unless-stopped \
      -p 192.168.56.20:3000:3000 bkimminich/juice-shop
ubuntu@cible:~$ sudo docker ps
CONTAINER ID  IMAGE                  STATUS        PORTS                          NAMES
a1b2c3d4e5f6  bkimminich/juice-shop  Up 2 minutes  192.168.56.20:3000->3000/tcp juice-shop
```

Élément	Rôle
`docker run`	Crée et démarre un conteneur
`-d`	Arrière-plan (detached)
`--name juice-shop`	Nom du conteneur
`--restart unless-stopped`	Redémarre le conteneur après un redémarrage de la machine, sauf arrêt volontaire
`-p 192.168.56.20:3000:3000`	Publie le port 3000 du conteneur sur le port 3000 de l'adresse privée **uniquement**
`bkimminich/juice-shop`	Image de l'application OWASP Juice Shop

À RETENIR Pourquoi préciser l'adresse dans -p ?

Sans elle (`-p 3000:3000`), Docker publie le port sur **toutes** les interfaces, y compris la carte NAT. En liant l'adresse privée, on garde l'application dans le laboratoire.

ATTENDU

Une ligne `juice-shop`, `Up …`, avec `192.168.56.20:3000->3000/tcp`. Au premier lancement, l'image est téléchargée depuis Internet : la carte NAT doit encore être active.

3.5 Configurer l'adresse de Kali

Sur Kali, repérez la carte Host-only avec `ip -br addr`, puis fixez son adresse avec NetworkManager :

kali@kali: ~

```
kali@kali:~$ nmcli connection show
kali@kali:~$ sudo nmcli connection modify "Wired connection 2" \
      ipv4.method manual ipv4.addresses 192.168.56.10/24
kali@kali:~$ sudo nmcli connection up "Wired connection 2"
kali@kali:~$ ip -br addr
```

PIÈGE

Le nom entre guillemets dépend de votre machine : reprenez celui de la liste affichée par `nmcli connection show`. L'adresse `192.168.56.10` est un exemple.

3.6 Vérifier la communication, couche par couche

On teste dans l'ordre des couches : **lien → réseau → transport → application**. Un test qui échoue indique la couche en cause.

kali@kali: ~

```
kali@kali:~$ ping -c 3 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=0.398 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=0.401 ms
3 packets transmitted, 3 received, 0% packet loss
kali@kali:~$ nc -zv 192.168.56.20 3000
192.168.56.20: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.56.20] 3000 (?) open
kali@kali:~$ curl -I http://192.168.56.20:3000/
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

Test	Couche	Attendu	Si échec
`ping -c 3`	Réseau (ICMP)	3 packets transmitted, 3 received	Vérifier adresses, même réseau Host-only, VM allumées. Le pare-feu peut aussi filtrer l'ICMP
`nc -zv IP 3000` (teste le port sans envoyer de données)	Transport (TCP)	`succeeded` ou `open`	Conteneur arrêté, mauvaise adresse de publication
`curl -I`	Application (HTTP)	`HTTP/1.1 200 OK` et des en-têtes	Application non démarrée ou erreur du service

À RETENIR

Si le ping échoue mais que `curl` fonctionne, le service est joignable malgré un filtrage ICMP : c'est un bon exemple de différence entre « la machine répond au ping » et « le service est accessible ».

Ouvrir l'application : dans le navigateur de Kali, saisissez `http://192.168.56.20:3000`. La page de la boutique Juice Shop doit s'afficher.

3.7 Isoler le laboratoire et prendre un instantané

1. Arrêtez les VM.
2. Dans la configuration de chaque VM, **désactivez l'adaptateur 1 (NAT)** (décochez « Activer l'interface réseau »).
3. Redémarrez : seule la carte Host-only reste.
4. Prenez un **instantané** (Machine → Prendre un instantané) nommé `labo-propre`. Il permet de revenir à un état sain après une manipulation.

RÈGLE R9 Garder un état sain de référence
Avant toute manipulation à risque, on prend un instantané. On peut ainsi revenir à un état connu et reproduire un test dans les mêmes conditions.

kali@kali: ~

```
kali@kali:~$ ping -c 2 8.8.8.8
ping: connect: Network is unreachable
kali@kali:~$ ping -c 2 192.168.56.20
2 packets transmitted, 2 received, 0% packet loss
```

ATTENDU Vérification
`ping -c 2 8.8.8.8` doit **échouer** (aucune route vers Internet) ; `ping -c 2 192.168.56.20` doit **réussir**.

3.8 Diagnostic des problèmes courants

Diagnostiquer, c'est raisonner : un message d'erreur indique une couche. **« refused »** signifie que la machine répond mais qu'aucun service n'écoute ; **« timed out »** signifie que rien ne répond (machine injoignable ou paquets filtrés).

Symptôme	Cause probable	Vérification
`Network is unreachable`	Aucune adresse sur la carte Host-only	`ip -br addr`
Ping sans réponse	VM éteinte, mauvais réseau VirtualBox, adresses dans des plages différentes	Même `vboxnet`, même /24 ; `ip route`
`Connection refused` sur le port 3000	Conteneur arrêté ou publié sur une autre adresse	`sudo docker ps`, `sudo docker logs juice-shop`
`Connection timed out`	Pare-feu ou mauvaise route	`sudo ufw status` sur la cible, `ip route`
`docker: command not found`	Docker non installé	Suivre la documentation Docker
Netplan refuse la configuration	Erreur d'indentation YAML	`sudo netplan try` indique la ligne fautive

PARTIE 4

Pratiquer : Nmap, Nikto, Gobuster

90 min

4

OBJECTIF
Pour chaque outil : principe → commande décomposée → résultat commenté → ce que l'on conclut → ce que l'on ne conclut pas.

ÉTAPE 4.0 Préparer l'arborescence de travail

```
kali@kali: ~
kali@kali:~$ mkdir -p ~/atelier-kali-1/{nmap,nikto,gobuster,wireshark,metadata,rapport}
kali@kali:~$ ls ~/atelier-kali-1
gobuster  metadata  nikto  nmap  rapport  wireshark
```

`mkdir -p` crée les dossiers (et les parents si besoin) ; les accolades `{a,b,c}` génèrent un dossier par nom. **Attendu :** six dossiers listés.

RÈGLE R12 Traçabilité
On conserve les commandes exactes, leurs sorties et la date : un résultat qu'on ne peut pas reproduire ni justifier n'a pas de valeur de preuve. Chaque outil enregistre donc ses résultats dans un fichier du dossier de travail.

4.1 Nmap — identifier les ports et les services

DÉFINITION Nmap
Outil de reconnaissance qui envoie des paquets vers une cible et déduit l'**état des ports** d'après les réponses. Il observe, il n'exploite pas.

ÉTAPE 4.1.1 Découvrir les hôtes du réseau (facultatif)

```
kali@kali: ~
kali@kali:~$ nmap -sn 192.168.56.0/24
Starting Nmap 7.9x ( https://nmap.org ) at 2026-10-21 10:12 CEST
Nmap scan report for 192.168.56.1
Host is up (0.00021s latency).
Nmap scan report for 192.168.56.10
Host is up (0.00008s latency).
Nmap scan report for 192.168.56.20
Host is up (0.00045s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.54 seconds
```

`-sn` = découverte d'hôtes uniquement, sans scan de ports. La sortie liste les adresses qui répondent (Kali, la cible, probablement l'interface de la machine hôte et le serveur DHCP de VirtualBox).

RAPPEL R1 Ne lancez cette commande que sur le réseau Host-only du labo

ÉTAPE 4.1.2 Scanner la cible de façon limitée

kali@kali: ~

```
kali@kali:~$ nmap -sV -T3 -p 22,80,3000 192.168.56.20
```

Paramètre	Signification
`nmap`	Le programme
`-sV`	Détection de version : Nmap interroge le service pour identifier le logiciel
`-T3`	Gabarit de temporisation « normal » (de `-T0` très lent à `-T5` très agressif)
`-p 22,80,3000`	Limite le scan à ces trois ports TCP
`192.168.56.20`	Cible

À RETENIR

Sans droits administrateur, Nmap utilise un scan par **connexion complète** (TCP connect). Avec `sudo`, il utilise par défaut un scan **SYN**, plus discret, qui n'achève pas la connexion. Les deux conviennent à l'atelier.

RÈGLE R11 Proportionnalité

On limite un scan à ce qui est nécessaire (ports ciblés, cadence modérée `-T3`). Un scan agressif charge le service, remplit les journaux et peut perturber un système en production.

ÉTAPE 4.1.3 Lire le résultat (illustratif)

résultat Nmap (illustratif — votre sortie peut différer)

```
Starting Nmap 7.9x ( https://nmap.org ) at 2026-10-21 10:15 CEST
Nmap scan report for 192.168.56.20
Host is up (0.00045s latency).

PORT     STATE  SERVICE VERSION
22/tcp   open   ssh     OpenSSH 9.x (Ubuntu Linux; protocol 2.0)
80/tcp   closed http
3000/tcp open   ppp?

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.30 seconds
```

Ligne	Lecture
`Nmap scan report for …`	Cible analysée
`Host is up (0.00045s latency)`	L'hôte a répondu ; le délai est très court, ce qui est normal entre deux VM
`22/tcp open ssh`	Un service accepte les connexions sur le port 22 et se présente comme SSH
`80/tcp closed http`	La machine répond mais **aucun service n'écoute** sur ce port. Le nom `http` vient de la table des ports usuels, pas d'une détection
`3000/tcp open ppp?`	Un service écoute. Le nom `ppp` provient de la table associant le numéro de port ; le **point d'interrogation** signale une identification incertaine

Les six états d'un port

État	Signification	Ce que Nmap a observé
`open`	Un service accepte les connexions	Réponse SYN/ACK
`closed`	Accessible mais aucun service n'écoute	Réponse RST
`filtered`	Impossible de savoir : un filtre bloque	Pas de réponse ou erreur ICMP
`unfiltered`	Accessible, ouvert/fermé indéterminé	Réponse à un scan ACK
`open|filtered`	Ouvert ou filtré (indécidable)	—
`closed|filtered`	Fermé ou filtré (indécidable)	—

ÉTAPE 4.1.4

Voir Nmap « de l'intérieur » avec Wireshark (démonstration)

Lancez une capture Wireshark sur l'interface Host-only, relancez le scan, puis appliquez ce filtre d'affichage :

filtre d'affichage Wireshark

```
ip.addr == 192.168.56.20 && tcp.port == 3000
```

Pour un **port ouvert**, vous devriez observer une séquence `SYN → SYN, ACK → ACK` (scan par connexion) ou `RST` (scan SYN). Pour le **port 80 fermé**, la cible répond `RST, ACK` immédiatement. Vous voyez ainsi concrètement d'où viennent les mots « open » et « closed ».

ÉTAPE 4.1.5

Enregistrer et relire

kali@kali: ~

```
kali@kali:~$ nmap -sV -T3 -p 22,80,3000 -oN ~/atelier-kali-1/nmap/scan.txt 192.168.56.20
kali@kali:~$ cat ~/atelier-kali-1/nmap/scan.txt
```

Option	Format
`-oN fichier`	Texte lisible (usage dans le rapport)
`-oX fichier`	XML (pour traitement par programme)
`-oG fichier`	Format « grepable » (ancien)
`-oA base`	Les trois formats à la fois

Options utiles

- `-p-` : scanne les 65535 ports TCP (plus long ; acceptable en labo).
- `-Pn` : ne teste pas si l'hôte répond au ping avant de scanner. Utile si le ping est filtré (`Host seems down`).
- `-v` : plus de détails pendant le scan.

Conclure

Élément	Formulation correcte
Observation	« Les ports 22/tcp et 3000/tcp sont ouverts ; le port 80/tcp est fermé »
Preuve	`scan.txt`, avec la commande exacte
Interprétation	« Un accès SSH et un service sur le port 3000 sont exposés dans le réseau de labo »
Ne pas conclure	« Le serveur est vulnérable » : une version n'est pas une faille tant qu'on n'a pas vérifié son applicabilité

4.2 Nikto — vérifier le serveur web

DÉFINITION Nikto
Scanner web qui envoie un grand nombre de requêtes HTTP prédéfinies pour détecter des fichiers connus, des versions, des configurations faibles et des en-têtes de sécurité absents. C'est un scanner **bruyant** : il laisse de nombreuses traces dans les journaux du serveur.

ÉTAPE 4.2.1 Lancer Nikto

kali@kali: ~

```
kali@kali:~$ nikto -host http://192.168.56.20:3000 -output ~/atelier-kali-1/nikto/nikto.txt
```

Paramètre	Signification
`-host URL`	Cible (adresse, port, schéma)
`-output fichier`	Enregistre le rapport
`-Format txt` (facultatif)	Choisit le format du fichier (txt, htm, csv, xml…)
`-Tuning` (facultatif)	Limite les familles de tests

ÉTAPE 4.2.2 Lire le résultat (illustratif)

résultat Nikto (illustratif — votre résultat peut différer)

```
- Nikto v2.x
+ Target IP:          192.168.56.20
+ Target Port:        3000
+ Server: No banner retrieved
+ /: The anti-clickjacking X-Frame-Options header is not present.
+ /: The X-Content-Type-Options header is not set.
+ /robots.txt: contains 1 entry which should be manually viewed.
+ 8000 requests: 0 error(s) and 3 item(s) reported on remote host
```

À RETENIR
Les lignes débutant par `+` sont des **constats à vérifier**, pas des preuves.

RÈGLE R6 Toute alerte se vérifie (rappel)
Chaque ligne de Nikto est une hypothèse. Le tableau ci-dessous indique comment la vérifier.

ÉTAPE 4.2.3 Vérifier chaque type d'alerte

Type d'alerte	Vérification	Remarque
En-tête de sécurité absent	`curl -I http://192.168.56.20:3000/` puis lire les en-têtes	Vérifier s'il est réellement absent, et son utilité dans ce contexte
Entrée dans `robots.txt`	`curl http://192.168.56.20:3000/robots.txt`	`robots.txt` est **public par nature** ; il indique des chemins, pas des secrets
Version de logiciel divulguée	Comparer la version aux avis de sécurité de l'éditeur	Une version ancienne n'est pas automatiquement vulnérable
Listage de répertoire	Ouvrir l'URL dans le navigateur	Vérifier le contenu réel des fichiers
Référence à une faille connue (CVE)	Lire l'avis, comparer version et configuration	Beaucoup de références ne s'appliquent pas à votre cible

lecture des en-têtes (illustratif : la réponse réelle peut différer de l'alerte Nikto)

```
kali@kali:~$ curl -I http://192.168.56.20:3000/
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Content-Type: text/html; charset=UTF-8
```

Dans cet exemple, un en-tête signalé « absent » par Nikto est en réalité **présent** : c'est la preuve qu'une alerte doit toujours être confrontée à une vérification manuelle.

Les en-têtes HTTP à connaître

En-tête	Rôle
`Server`, `X-Powered-By`	Révèlent le logiciel serveur (information technique)
`Content-Type`	Type du contenu renvoyé
`Set-Cookie`	Création d'un cookie ; vérifier les attributs `Secure`, `HttpOnly`, `SameSite`
`X-Frame-Options`	Limite l'inclusion de la page dans une autre (clickjacking)
`X-Content-Type-Options`	`nosniff` empêche le navigateur de deviner le type
`Content-Security-Policy`	Restreint les sources de scripts et de contenus
`Strict-Transport-Security`	Impose HTTPS ; n'a de sens qu'en HTTPS

PIÈGE
Signaler l'absence de `Strict-Transport-Security` sur un service en **HTTP** est une consigne de bon sens, mais l'en-tête n'a aucun effet tant que le site n'est pas servi en HTTPS.

PIÈGE
Juice Shop est une **application monopage**. Certains tests de Nikto peuvent obtenir des réponses « 200 » génériques et signaler des fichiers qui n'existent pas. D'où la règle : toute alerte se vérifie.

4.3 Gobuster — découvrir des chemins web

DÉFINITION Énumération par dictionnaire
Technique qui teste, pour chaque mot d'une liste, si l'URL `http://cible/mot` existe, et note le code de réponse. Gobuster ne trouve **que** ce qui figure dans la liste.

ÉTAPE 4.3.1 Créer une liste de mots

kali@kali: ~

```
kali@kali:~$ cat > ~/atelier-kali-1/paths.txt <<'EOF'
robots.txt
sitemap.xml
admin
backup
uploads
assets
ftp
EOF
kali@kali:~$ cat ~/atelier-kali-1/paths.txt
```

`cat > fichier <<'EOF'` écrit les lignes suivantes dans le fichier jusqu'à la ligne `EOF`. Les guillemets autour de `EOF` empêchent le shell d'interpréter le contenu.

ÉTAPE 4.3.2 Lancer l'énumération

kali@kali: ~

```
kali@kali:~$ gobuster dir -u http://192.168.56.20:3000/ \
      -w ~/atelier-kali-1/paths.txt -t 10 \
      -o ~/atelier-kali-1/gobuster/dirs.txt
```

Le caractère `\` en fin de ligne indique que la commande continue sur la ligne suivante.

Paramètre	Signification
`dir`	Mode découverte de répertoires et fichiers
`-u`	URL de base
`-w`	Liste de mots
`-t 10`	10 tâches en parallèle (augmenter charge le serveur)
`-o`	Fichier de sortie
`-x txt,bak` (facultatif)	Ajoute des extensions à chaque mot
`-s` / `-b` (facultatif)	Codes de statut à retenir / à exclure

ÉTAPE 4.3.3 Lire le résultat (illustratif)

résultat Gobuster (illustratif)

```
/ftp                 (Status: 200) [Size: 11062]
/robots.txt          (Status: 200) [Size: 28]
/assets              (Status: 301) [Size: 179] [--> /assets/]
```

Code HTTP	Signification	Conduite
`200`	Ressource accessible	Examiner le contenu
`301`, `302`	Redirection	Suivre la redirection
`401`	Authentification requise	Constat : zone protégée
`403`	Accès interdit	Refus : **pas une permission de contourner**
`404`	Ressource absente	Normal pour un chemin inexistant
`500`	Erreur serveur	Noter, à analyser

ÉTAPE 4.3.4 Valider un résultat avec curl

kali@kali: ~

```
kali@kali:~$ curl -sS -o /dev/null -w '%{http_code} %{content_type} %{size_download}\n' \
      http://192.168.56.20:3000/robots.txt
200 text/plain; charset=UTF-8 28
kali@kali:~$ curl -sS -D - -o /dev/null http://192.168.56.20:3000/backup
```

Option	Rôle
`-s`	Silencieux (pas de barre de progression)
`-S`	Affiche tout de même les erreurs
`-o /dev/null`	Jette le corps de la réponse
`-w '…'`	Affiche des variables après la requête (code, type, taille)

Option	Rôle
`-D -`	Affiche les en-têtes de réponse
`-I`	Envoie une requête HEAD (en-têtes seuls)
`-i`	Inclut les en-têtes dans la sortie
`-L`	Suit les redirections

ÉTAPE 4.3.5 — Repérer un faux positif

Si Gobuster signale presque tous les chemins avec la **même taille**, l'application renvoie probablement une page générique. Testez un chemin qui n'existe certainement pas :

kali@kali: ~ (cas d'un faux positif : 200 générique)

```
kali@kali:~$ curl -sS -o /dev/null -w '%{http_code} %{size_download}\n' \
      http://192.168.56.20:3000/zz-inexistant-4821
200 75002
```

- Si la réponse est un **200 de même taille** que celle des chemins « trouvés » : ces résultats sont des **faux positifs**.
- Si c'est un **404** : les 200 ont plus de chances d'être réels, mais il faut encore en vérifier le contenu.
- Les versions récentes de Gobuster signalent ce cas d'elles-mêmes et proposent d'exclure un code ou une taille (option `--exclude-length` selon la version ; consultez `gobuster dir --help`).

NE PAS CONCLURE

Qu'un chemin découvert contient des données sensibles. `/assets` est un dossier de ressources normales ; `/ftp` doit être **ouvert et lu** avant toute classification.

BONNE PRATIQUE

Une énumération génère des centaines ou milliers de requêtes. En situation réelle (avec autorisation), on la planifie avec le responsable du système, car elle est visible dans les journaux et peut charger le service.

PARTIE 5

Pratiquer : capture et métadonnées

60 min

5

OBJECTIF
Capturer un échange HTTP dans le laboratoire, le relire avec Wireshark et `tshark`, constater qu'il est lisible en clair, puis examiner les métadonnées d'un fichier avec ExifTool.

5.1 Wireshark en situation : capturer un échange HTTP

RÈGLE R15 Protéger les captures
Une capture (`.pcapng`) peut contenir des identifiants, des cookies, des données personnelles. Elle est stockée dans le dossier de travail du labo, jamais envoyée par messagerie ni publiée, et supprimée une fois l'exercice terminé.

ÉTAPE 5.1.1 Identifier l'interface Host-only

kali@kali: ~

```
kali@kali:~$ ip -br addr
lo               UNKNOWN    127.0.0.1/8
eth0             DOWN
eth1             UP         192.168.56.10/24
kali@kali:~$ tshark -D
1. eth1
2. eth0
3. any
4. lo (Loopback)
```

PIÈGE
Capturer sur la mauvaise interface : aucun paquet attendu n'apparaît. Le trafic Kali ↔ cible passe par la carte **Host-only** (ici `eth1`), pas par `lo`.

ÉTAPE 5.1.2 Démarrer la capture avec un filtre de capture

Dans Wireshark : **Capture → Options**, sélectionner `eth1`, champ « Filtre de capture » : `host 192.168.56.20 and port 3000`, puis **Démarrer**. L'équivalent en ligne de commande :

kali@kali: ~

```
kali@kali:~$ tshark -i eth1 -f "host 192.168.56.20 and port 3000" -a duration:30 \
      -w ~/atelier-kali-1/wireshark/httptest.pcapng
Capturing on 'eth1'
(laisser tourner : on génère du trafic dans un second terminal)
```

PIÈGE
Un filtre de **capture** erroné fait perdre définitivement des paquets (ils ne sont pas enregistrés). En cas de doute, capturez sans filtre et filtrez à l'affichage.

ÉTAPE 5.1.3 Générer du trafic (second terminal)

kali@kali: ~ (second terminal)

```
kali@kali:~$ curl -i http://192.168.56.20:3000/robots.txt
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 28

User-agent: *
Disallow: /ftp
kali@kali:~$ curl -sS -X POST http://192.168.56.20:3000/rest/user/login \
      -H 'Content-Type: application/json' \
      -d '{"email":"demo@labo.test","password":"faux-mot-de-passe-1"}'
Invalid email or password.
```

> **RÈGLE R2 Données fictives (rappel)**
> Les identifiants ci-dessus sont **inventés**. On ne saisit jamais un vrai mot de passe dans un test de capture.

ÉTAPE 5.1.4 Arrêter, enregistrer, relire

Arrêtez la capture (carré rouge) et enregistrez sous `~/atelier-kali-1/wireshark/httptest.pcapng`. Relisez-la en ligne de commande :

résultat tshark (illustratif)

```
kali@kali:~$ tshark -r ~/atelier-kali-1/wireshark/httptest.pcapng -Y http
  4 0.000412 192.168.56.10 → 192.168.56.20 HTTP 143 GET /robots.txt HTTP/1.1
  6 0.003120 192.168.56.20 → 192.168.56.10 HTTP 391 HTTP/1.1 200 OK (text/plain)
 12 5.210874 192.168.56.10 → 192.168.56.20 HTTP 291 POST /rest/user/login HTTP/1.1 (application/json)
 14 5.226310 192.168.56.20 → 192.168.56.10 HTTP 322 HTTP/1.1 401 Unauthorized (text/html)
```

ÉTAPE 5.1.5 Observer le début de la connexion TCP

filtre d'affichage Wireshark

```
tcp.flags.syn == 1
```

Vous devez voir, pour chaque connexion, `[SYN]` puis `[SYN, ACK]` puis `[ACK]` : la poignée de main en trois temps vue en partie 1.

ÉTAPE 5.1.6 Suivre le flux TCP : constater le clair

Dans Wireshark : clic droit sur un paquet HTTP → **Suivre → Flux TCP**. Équivalent en ligne de commande :

flux TCP reconstitué (illustratif)

```
kali@kali:~$ tshark -r ~/atelier-kali-1/wireshark/httptest.pcapng -q -z follow,tcp,ascii,1
===================================================================
Follow: tcp,ascii
Node 0: 192.168.56.10:51234
Node 1: 192.168.56.20:3000
POST /rest/user/login HTTP/1.1
Host: 192.168.56.20:3000
Content-Type: application/json

{"email":"demo@labo.test","password":"faux-mot-de-passe-1"}
```

ATTENDU

L'adresse e-mail **et** le mot de passe (fictifs) apparaissent en clair dans la capture, parce que la connexion est en HTTP et non en HTTPS.

À RETENIR **Ce que cela démontre**

Toute personne capable d'observer ce segment réseau peut lire ce qui y transite en HTTP. C'est la raison d'être de la règle R3 (chiffrer les échanges).

NE PAS CONCLURE

Que Juice Shop est « piratable » : on a seulement constaté qu'un échange HTTP est lisible. L'authentification a d'ailleurs bien refusé ces identifiants fictifs.

5.2 ExifTool : les métadonnées

DÉFINITION **Métadonnées**

Données qui décrivent un fichier sans en faire partie visible : auteur, logiciel, appareil, date, parfois **position GPS** (EXIF dans les images). Elles voyagent avec le fichier, même si l'on ne les voit pas à l'écran.

ÉTAPE 5.2.1 — Récupérer un fichier du labo

kali@kali: ~

```
kali@kali:~$ curl -sS -o ~/atelier-kali-1/metadata/apple_juice.jpg \
      http://192.168.56.20:3000/assets/public/images/products/apple_juice.jpg
kali@kali:~$ file ~/atelier-kali-1/metadata/apple_juice.jpg
apple_juice.jpg: JPEG image data, JFIF standard 1.01, 1000x1000
```

PIÈGE

Le chemin de l'image dépend de la version de Juice Shop. Si le téléchargement échoue (`404`), repérez un nom d'image dans le navigateur (clic droit → copier l'adresse de l'image).

ÉTAPE 5.2.2 — Lire les métadonnées

résultat ExifTool (illustratif)

```
kali@kali:~$ exiftool ~/atelier-kali-1/metadata/apple_juice.jpg
ExifTool Version Number         : 12.xx
File Name                       : apple_juice.jpg
File Size                       : 71 kB
File Type                       : JPEG
MIME Type                       : image/jpeg
Image Width                     : 1000
Image Height                    : 1000
```

ÉTAPE 5.2.3 — Ajouter des métadonnées fictives pour la démonstration

```
kali@kali: ~
kali@kali:~$ exiftool -Artist="Etudiant Test" -Copyright="Labo BTS SIO" -Comment="demo atelier" \
      ~/atelier-kali-1/metadata/apple_juice.jpg
    1 image files updated
kali@kali:~$ exiftool -a -G1 -s ~/atelier-kali-1/metadata/apple_juice.jpg | grep -E "Artist|Copyright|Comm
[IFD0]          Artist                          : Etudiant Test
[IFD0]          Copyright                       : Labo BTS SIO
[File]          Comment                         : demo atelier
```

Option	Rôle
`-a`	Affiche aussi les champs en double
`-G1`	Indique le groupe de chaque champ (EXIF, IPTC, XMP…)
`-s`	Noms de champs courts
`-gps:all`	Affiche uniquement les champs de géolocalisation
`-all=`	Efface toutes les métadonnées (crée une copie `_original`)
`-overwrite_original`	N'enregistre pas de copie de sauvegarde

ÉTAPE 5.2.4 Cas réel : une photo de smartphone

Une photo prise avec un téléphone contient souvent la position, la date et le modèle de l'appareil :

```
exemple fictif — ne jamais utiliser la photo d'un tiers
kali@kali:~$ exiftool -gps:all -Make -Model -DateTimeOriginal photo_telephone.jpg
Make                            : ExampleCorp
Model                           : Phone X
Date/Time Original              : 2026:10:14 18:42:07
GPS Latitude                    : 48 deg 51' 24.00" N
GPS Longitude                   : 2 deg 21' 07.00" E
```

NE PAS CONCLURE

Qu'une position GPS lue dans une image est celle du lieu actuel de la personne : c'est le lieu de **prise de vue**, à la date indiquée, et la valeur peut être modifiée.

ÉTAPE 5.2.5 Nettoyer avant de publier

```
kali@kali: ~
kali@kali:~$ exiftool -all= -overwrite_original ~/atelier-kali-1/metadata/apple_juice.jpg
    1 image files updated
kali@kali:~$ exiftool ~/atelier-kali-1/metadata/apple_juice.jpg | grep -E "Artist|GPS"
(aucune ligne : les métadonnées ont été supprimées)
```

RÈGLE R16 Nettoyer les métadonnées avant diffusion

Avant de publier une image ou un document, on supprime les métadonnées inutiles (position, auteur, logiciel, historique). Elles peuvent révéler un lieu, une identité ou un outil interne.

PARTIE 6

Analyser et rédiger

45 min

6

OBJECTIF
Transformer des sorties d'outils en conclusions solides : distinguer observation, preuve, faux positif, exposition potentielle et vulnérabilité confirmée, puis évaluer le risque et rédiger un rapport.

6.1 De l'observation à la recommandation

1. **Observer** : relever le fait brut (« le port 3000/tcp est ouvert »).
2. **Prouver** : associer un élément vérifiable (`scan.txt`, commande, date).
3. **Vérifier** : confronter l'alerte à un second moyen (curl, lecture du contenu, documentation).
4. **Qualifier** : faux positif, exposition potentielle ou vulnérabilité confirmée ?
5. **Évaluer le risque** : vraisemblance × impact.
6. **Recommander** : une action concrète, proportionnée, réalisable.

6.2 Évaluer un risque

Le risque combine **vraisemblance** et **impact**. On les place sur une matrice à trois niveaux :

Vraisemblance ↓ / Impact →	Faible	Moyen	Élevé
Élevée	**Moyen**	**Élevé**	**Critique**
Moyenne	**Faible**	**Moyen**	**Élevé**
Faible	**Faible**	**Faible**	**Moyen**

À RETENIR Exemple
Port 22 (SSH) ouvert dans le labo isolé : vraisemblance **faible** (réseau privé), impact **moyen** → risque **faible**. Le même port exposé sur Internet avec un mot de passe faible : vraisemblance **élevée**, impact **élevé** → risque **critique**. Le contexte change tout.

6.3 Distinguer un faux positif : la démarche

Question	Si oui	Si non
L'alerte est-elle reproductible à la main (curl, navigateur) ?	Passer à la question suivante	**Faux positif probable**
Le contenu renvoyé est-il réellement celui annoncé (et pas une page générique) ?	Passer à la question suivante	**Faux positif**
Le contenu est-il sensible (données personnelles, secrets, configuration) ?	**Exposition réelle** : qualifier le risque	Exposition sans gravité : noter
La version ou la configuration est-elle effectivement concernée par la faille citée ?	**Vulnérabilité confirmée** (en contexte autorisé)	Exposition potentielle seulement

RÈGLE R14 Défense en profondeur
On ne s'appuie jamais sur une seule protection : pare-feu, mises à jour, authentification, chiffrement, journalisation et supervision se complètent. Si une couche cède, les autres limitent les dégâts.

6.4 Rédiger le rapport

Section	Contenu attendu
1. Contexte et périmètre	Objectif, machines ciblées (`192.168.56.20`), date, autorisation (ici : cadre pédagogique)
2. Méthode et outils	Outils, versions, **commandes exactes** utilisées
3. Résultats	Tableau des constats : observation, preuve, statut
4. Analyse	Faux positifs écartés, expositions potentielles, vulnérabilités confirmées, risque
5. Recommandations	Actions priorisées, proportionnées
6. Limites	Ce qui n'a pas été testé ; ce que l'on ne peut pas conclure
Annexes	Fichiers `scan.txt`, `nikto.txt`, `dirs.txt`, `httptest.pcapng`

Exemple de tableau de constats (illustratif)

N°	Observation	Preuve	Statut	Risque	Recommandation
F1	Port 3000/tcp ouvert (application web)	`nmap/scan.txt`	Confirmé	Faible (labo)	Ne publier que sur l'interface privée
F2	En-tête `X-Frame-Options` signalé absent par Nikto	`nikto.txt` + `curl -I`	**Faux positif** (présent)	Nul	Aucune
F3	Chemin `/ftp` répondant 200	`gobuster/dirs.txt`	À examiner (contenu à lire)	À qualifier	Lire le contenu puis classer
F4	Échange HTTP lisible en clair	`httptest.pcapng`	Confirmé	Moyen (si prod)	Servir en HTTPS (R3)

> **RÈGLE R17 Divulgation responsable**
> Si l'on découvre une faille dans un système réel, on la signale **uniquement** au responsable du système, de façon discrète et factuelle, sans l'exploiter, sans en extraire de données et sans la rendre publique avant correction.

6.5 Grille d'évaluation

Critère	Points	Attendu
Respect du cadre légal et du périmètre	3	Aucune cible hors labo ; données fictives
Installation et vérifications	3	Chaque étape validée (ping, nc, curl, docker ps)
Mise en œuvre des outils	4	Commandes correctes, options comprises, résultats enregistrés
Preuves	3	Fichiers et commandes exactes référencés
Analyse : faits / hypothèses / faux positifs	4	Aucune conclusion abusive ; alertes vérifiées
Recommandations	2	Concrètes et proportionnées
Qualité de la rédaction	1	Clarté, structure
Total	**20**	

PARTIE 7

Appliquer : exercices

60 min

7

7.1 Exercices guidés

Exercice A — Lire un scan Nmap.

sortie Nmap fournie

```
PORT     STATE    SERVICE
21/tcp   filtered ftp
22/tcp   open     ssh
80/tcp   closed   http
443/tcp  open     https
3000/tcp open     ppp?
```

- Donnez l'état de chaque port et ce que Nmap a observé.
- Rédigez une observation correcte.
- Que ne peut-on pas conclure du `ppp?` ?

Exercice B — Classer des alertes Nikto. Pour chaque alerte, indiquez la vérification à faire : (1) `X-Frame-Options` absent ; (2) `/robots.txt` contient 1 entrée ; (3) `Server: Apache/2.2.x` ; (4) `/admin/` trouvé.

Exercice C — Gobuster. Gobuster signale 7 chemins en `Status: 200 [Size: 75002]`, et `curl` sur `/zz-inexistant-4821` renvoie `200 75002`. Quelle est votre conclusion ?

Exercice D — Filtres Wireshark. Écrivez le filtre d'affichage qui retient : (1) les requêtes HTTP POST ; (2) les connexions TCP refusées ; (3) tout le trafic de `192.168.56.10` vers le port 3000.

Exercice E — ExifTool. Une image publiée par une association affiche `GPS Latitude/Longitude` et `Artist`. Quel est le risque et quelle action recommandez-vous ?

Corrigés des exercices guidés

A. 21 : `filtered` (aucune réponse ou erreur ICMP, un filtre bloque) ; 22 : `open` (SYN/ACK) ; 80 : `closed` (RST) ; 443 : `open` ; 3000 : `open`. Observation : « Les ports 22, 443 et 3000 sont ouverts ; 80 est fermé ; 21 est filtré. » Le `?` signale une identification **incertaine** : on sait qu'un service répond, pas lequel.

B. (1) `curl -I` : l'en-tête est-il réellement absent ? (2) `curl` sur `/robots.txt` : fichier public, indique des chemins, pas des secrets ; (3) comparer la version aux avis de sécurité ; une version ancienne n'est pas automatiquement vulnérable ; (4) ouvrir l'URL et lire le contenu réel.

C. Il s'agit de **faux positifs** : l'application renvoie la même page générique pour des chemins inexistants. Il faut exclure cette taille (`--exclude-length`) ou valider chaque chemin à la main.

D. (1) `http.request.method == "POST"` ; (2) `tcp.flags.reset == 1` ; (3) `ip.src == 192.168.56.10 && tcp.dstport == 3000`.

E. Risque : révéler un **lieu** et une **identité**. Action : supprimer les métadonnées (`exiftool -all= -overwrite_original`) avant publication et republier l'image nettoyée (R16).

7.2 Exercices autonomes

1. **Mini-rapport.** Reproduisez le scan Nmap, l'énumération Gobuster et la capture HTTP, puis rédigez un rapport d'une page suivant la structure du 6.4, avec au moins un faux positif écarté.
2. **Comparaison de scans.** Lancez `nmap -sT` puis `sudo nmap -sS` sur le port 3000 et comparez les paquets dans Wireshark. Expliquez la différence observée.
3. **Du clair au chiffré.** Décrivez, sans la mettre en œuvre, comment servir Juice Shop en HTTPS et ce qu'un observateur verrait alors dans Wireshark.

Éléments attendus

- **1.** Tous les fichiers de preuve référencés ; distinction faits/hypothèses ; recommandations proportionnées.
- **2.** Le scan connexion (`-sT`) achève la poignée de main puis ferme ; le scan SYN (`-sS`) répond `RST` au `SYN, ACK` sans établir la connexion.
- **3.** Un certificat TLS et un point d'entrée HTTPS (443) ; l'observateur voit des octets illisibles au lieu du texte HTTP, mais voit toujours les adresses IP et la taille des échanges.

PARTIE 8

Réviser

30 min

8

8.1 Récapitulatif des règles de cybersécurité

Règle	Énoncé
R1	Autorisation préalable et périmètre
R2	Données fictives et confidentialité
R3	Chiffrer les échanges
R4	Réduire la surface d'exposition
R5	Séparer faits, hypothèses et recommandations
R6	Toute alerte se vérifie
R7	Moindre privilège
R8	Isoler le laboratoire
R9	Garder un état sain de référence (instantané)
R10	Vérifier l'intégrité de ce que l'on télécharge
R11	Proportionnalité des scans
R12	Traçabilité
R13	Changer les secrets par défaut
R14	Défense en profondeur
R15	Protéger les captures
R16	Nettoyer les métadonnées avant diffusion
R17	Divulgation responsable

8.2 Quiz corrigé

1. Quelle adresse identifie une carte réseau sur le lien local ?

A. Adresse IP
B. Adresse MAC
C. Port
D. Nom DNS

2. Que contient la MAC de destination d'une trame vers un serveur Internet ?

A. La MAC du serveur
B. `ff:ff:ff:ff:ff:ff`
C. La MAC du routeur
D. `00:00:00:00:00:00`

3. Quelle valeur du champ Type d'Ethernet annonce IPv4 ?

A. `0x0806`
B. `0x0800`
C. `0x86DD`
D. `0x0600`

4. Un port qui répond `RST` est :

A. open
B. filtered
C. closed
D. unfiltered

5. Combien d'octets d'en-tête Ethernet voit-on dans Wireshark ?

A. 8
B. 14
C. 18
D. 20

6. Le TTL de 118 prouve que la cible est un Windows à 10 sauts.

A. Vrai
B. Faux : simple indication

7. Un filtre de capture mal écrit :

A. Masque des paquets
B. Fait perdre des paquets définitivement
C. Est sans effet
D. Ralentit seulement

8. Une alerte Nikto est :

A. Une preuve
B. Une hypothèse à vérifier
C. Une vulnérabilité confirmée
D. Un faux positif

9. Pourquoi Docker est-il lancé avec `-p 192.168.56.20:3000:3000` ?

A. Pour accélérer
B. Pour ne publier le port que sur la carte privée
C. Pour activer HTTPS
D. Pour ouvrir le port 22

10. Que démontre un flux HTTP lisible dans Wireshark ?

A. Que le serveur est piraté
B. Que le contenu transite en clair
C. Que le pare-feu est absent
D. Rien

11. Quel outil efface les métadonnées d'une image ?

A. Nmap
B. Gobuster
C. ExifTool
D. Nikto

12. Un `403` signifie :

A. Ressource absente
B. Accès interdit : pas une permission de contourner
C. Redirection
D. Succès

Corrigé du quiz

Q	1	2	3	4	5	6	7	8	9	10	11	12
Réponse	B	C	B	C	B	B	B	B	B	B	C	B

8.3 Lexique

Terme	Définition
ARP	Protocole qui associe une adresse IP à une adresse MAC sur le lien local
Bannière	Texte de présentation renvoyé par un service
BPF	Syntaxe des filtres de capture
CVE	Identifiant public d'une vulnérabilité connue
DNS	Système de résolution des noms en adresses IP
Encapsulation	Ajout d'un en-tête par chaque couche
Énumération	Découverte de ressources par dictionnaire
Faux positif	Alerte d'outil inexacte
FCS	Contrôle d'erreur de fin de trame
Host-only	Mode réseau VirtualBox limité à l'hôte et aux VM
ICMP	Protocole de contrôle et de diagnostic (ping)
MAC / OUI	Adresse de carte réseau / identifiant du constructeur
MTU	Taille maximale de la charge utile (1500 octets en Ethernet)
Métadonnées	Données décrivant un fichier
Port éphémère	Port source choisi temporairement par le client
STP	Protocole évitant les boucles entre commutateurs
TCP / UDP	Transport avec / sans connexion
TLS	Chiffrement des échanges (base de HTTPS)
TTL	Durée de vie d'un paquet IP
Vulnérabilité	Faiblesse exploitable, confirmée en contexte

8.4 Fiche mémo des commandes

Objectif	Commande
Découvrir les hôtes	`nmap -sn 192.168.56.0/24`
Scanner des ports + versions	`nmap -sV -T3 -p 22,80,3000 192.168.56.20 -oN scan.txt`
Scanner le web	`nikto -host http://192.168.56.20:3000 -output nikto.txt`
Énumérer des chemins	`gobuster dir -u URL -w liste.txt -t 10 -o dirs.txt`
Lire les en-têtes	`curl -I http://192.168.56.20:3000/`
Tester un chemin	`curl -sS -o /dev/null -w '%{http_code} %{size_download}\n' URL`
Capturer	`tshark -i eth1 -f "host 192.168.56.20 and port 3000" -w fichier.pcapng`
Relire HTTP	`tshark -r fichier.pcapng -Y http`
Lire les métadonnées	`exiftool fichier.jpg`
Effacer les métadonnées	`exiftool -all= -overwrite_original fichier.jpg`

8.5 FAQ

Mon ping échoue mais curl fonctionne : est-ce grave ? Non. Le pare-feu filtre peut-être l'ICMP ; le service reste joignable.

Nmap affiche `ppp?` pour le port 3000 : est-ce une erreur ? Non. Le nom vient d'une table de ports ; le `?` indique une identification incertaine. Retenez : un service répond.

Wireshark n'affiche aucun paquet. Vérifiez l'interface choisie, le filtre de capture et le filtre d'affichage actif.

Gobuster trouve tout : que faire ? Testez un chemin inexistant : si la réponse est identique, ce sont des faux positifs ; excluez la taille.

Puis-je tester le site de mon établissement ? Non, sauf autorisation écrite du responsable. Cadre légal : articles 323-1 et suivants du Code pénal (R1).

8.6 Documentation

Sujet	Source
Nmap	nmap.org/book/
Wireshark	wireshark.org/docs/
Kali Linux	kali.org/docs/
OWASP Juice Shop	pwning.owasp-juice.shop
Docker (installation sur Ubuntu)	docs.docker.com/engine/install/ubuntu/
Sécurité des systèmes d'information	cyber.gouv.fr (ANSSI)
Données personnelles	cnil.fr
Textes de loi	legifrance.gouv.fr (articles 323-1 et suivants du Code pénal)

> **À RETENIR** **Pour conclure**
> Observer, prouver, vérifier, qualifier, recommander — **toujours dans le cadre autorisé**. C'est la différence entre un outil lancé « pour voir » et une démarche d'analyse de sécurité.